

UNIVERSITI TUNKU ABDUL RAHMAN

ACADEMIC YEAR 2025/2026

UECS3253 WIRELESS APPLICATION DEVELOPMENT**RECIPE TAKING APPLICATION**

Group Name:			
Student Name	Student ID	Practical Group	Programme
Kau Yong Jun	2205053	P4	SE
Chong Poh Yeet	2500645	P1	SE
Chong Wei Xu	2500549	P6	SE
Amos Khew Ze Hang	2500227	P2	SE

PART	CO	Component	Criteria	Marks Allocated
A	1	Report	Screens & Layout	/5
	2		Use of Appropriate Components	/5
	3		Reusability (Custom Components) inserts/updates	/5
	4		Implementation of React-Navigation: Stack, Drawer, Tab	/10
	5	Demonstration	User Interface	/5
Subtotal				/30
B	1	Report	Appropriate Usage of Data Persistence	/10
	2		Implementation of Data Persistence	/10
	3		CRUD Operations & input validation	/10
	4	Demonstration	CRUD and Data Persistence	/5
Subtotal				/35
C	1	Report	Appropriate Usage of Cloud Connectivity	/15
	2		Implementation of Cloud Connectivity	/15
	3	Demonstration	Cloud Connectivity	/5
Subtotal				/35

[Total: /100 marks]

Lee Kong Chian Faculty of Engineering and Science

Universiti Tunku Abdul Rahman

TABLE OF CONTENTS

PART

A	SCREENS, COMPONENTS AND REACT-NAVIGATION	5
	A.1 Screens & Layout	5
	A.2 Use of Components	14
	A.3 Reusability (Custom Components)	16
	A.4 Implementation of React-Navigation	22
B	DATA PERSISTENCE	27
	B.1 Usage of Data Persistence	27
	B.2 Implementation of Data Persistence	28
	B.3 CRUD Operations & Input Validation	35
C	CLOUD CONNECTIVITY	48
	C.1 Usage of Cloud Connectivity	48
	C.2 Implementation of Cloud Connectivity	
	Error! Bookmark not defined.0	
	REFERENCES	59
	PROJECT LINK	60

BRIEF SUMMARY

The Recipe Manager App is a mobile application built using React Native, targeting the Android platform. React Native is an open-source framework developed by Meta that allows developers to build native mobile applications using JavaScript and React (React Native, 2026A). The app is designed to help users organise, browse, and manage their personal collection of culinary recipes. The application is structured around a three-layer navigation hierarchy, Stack, Drawer, and Tab navigators, allowing users to seamlessly move between the recipe library, individual recipe details, editing forms, and utility screens. All screens share a consistent header-and-content layout pattern and a unified colour palette, giving the app a cohesive visual identity.

On the data layer, recipe records are stored locally in a SQLite relational database managed through the react-native-sqlite-storage library, while the user's list of favourite recipes persisted independently using AsyncStorage key-value storage. AsyncStorage is described as an unencrypted, asynchronous, persistent, key-value storage system that is global to the app (React Native, 2026B). This separation ensures that toggling a favourite is lightweight and does not require a full SQL write operation. For user-controlled backups, the app integrates the react-native-fs library to export and import JSON snapshots of all recipe and favourites data directly to the device's file system.

For cloud connectivity, the app communicates with a custom Node.js Express server over both RESTful Web API calls and WebSocket connections powered by Socket.IO, enabling real-time cloud backup and restore functionality. Node.js is a cross-platform, open-source JavaScript runtime environment that can run on servers and various operating systems (OpenJS Foundation, 2026). The WebSocket channel provides immediate acknowledgement and status feedback during backup and restore operations, while the REST API handles stateless queries such as fetching the last backup timestamp on screen load. Together, these layers make the Recipe Manager App a fully offline-capable application with optional cloud and local backup safety nets.

KEY FEATURES

- **Recipe Library:** Browse all saved recipes in a scrollable card list, each displaying the recipe title, preparation time, calorie count, and star rating.
- **Add, Edit, and Delete Recipes:** Full CRUD operations are available, allowing users to create new recipes with fields for title, description, ingredients, cooking instructions, preparation time, image URL, calories, and rating. Input validation is enforced before saving.
- **Favourites System:** Users can mark or unmark any recipe as a favourite with a heart toggle, persisted via AsyncStorage. A dedicated Favourites tab shows only the favourited recipes.
- **Cloud Backup and Restore (WebSocket):** The Cloud Sync screen allows users to back up all local data to a cloud server using Socket.IO WebSockets, and to restore it back at any time.
- **Local File Backup:** The File Backup screen allows users to export all recipe and favourites data as a JSON file to device storage, and import it back whenever needed.
- **About Screen:** Displays app information, version, feature list, and developer credits.

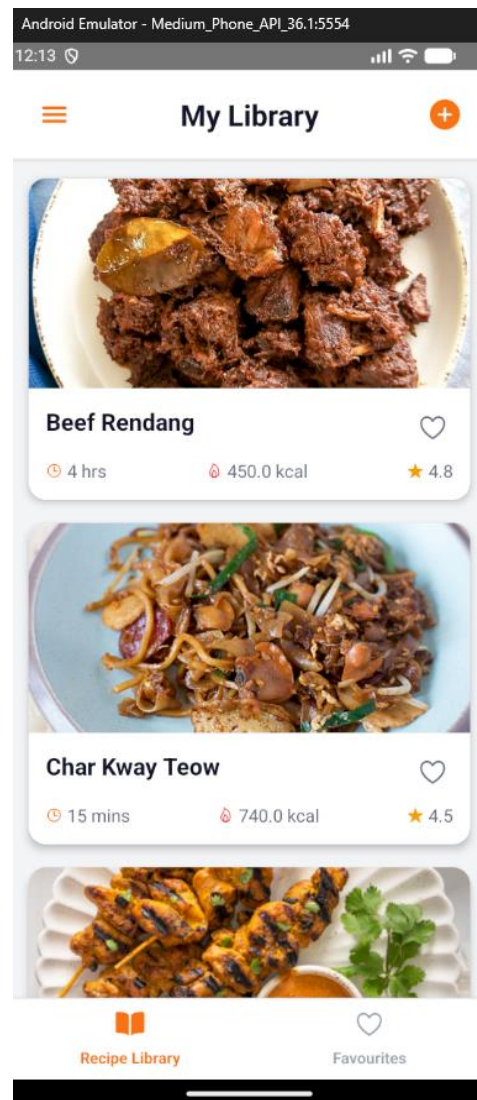
A SCREENS, COMPONENTS AND REACT-NAVIGATION

A.1 Screens & Layout

The Recipe Manager App is composed of eight distinct screens, each fulfilling a specific role within the application workflow. According to React Native (2026C), React Native components map directly to the platform's native UI building blocks, ensuring that interfaces feel natural on Android. All screens follow a consistent structural pattern: a custom header bar at the top contains a left-side icon (hamburger menu or back arrow), a centred title Text, and an optional right-side action icon. Below the header, the main content area uses View, ScrollView, or FlatList depending on the screen's purpose. This design approach is consistent with React Native's component-based model, where each UI element is a reusable, composable piece of the interface (React Native, 2026C).

The eight screens and their individual layout characteristics are described below:

- **RecipeLibraryScreen:** The first tab screen in the Bottom Tab Navigator, serving as the app's home screen. It renders a FlatList of recipe cards. Each card uses a View container with elevation: 4 for shadow, a full-width Image for the recipe photo, and a content View below containing a title row (recipe title and heart TouchableOpacity for favourites) and a detail row with three icon-text pairs showing prep time, calories, and rating. The header shows "My Library" and an add-circle icon on the right.



RecipeLibraryScreen — recipe card list with image, title, prep time, calories, rating, and heart icon

Figure A.1 – Recipe Library Screen

- **FavouriteScreen:** The second tab screen in the Bottom Tab Navigator, structurally identical to RecipeLibraryScreen, reusing the RecipeList custom component. It only displays recipes whose IDs are present in the AsyncStorage favourites array. When the list is empty, a styled emptyText paragraph is shown instead.

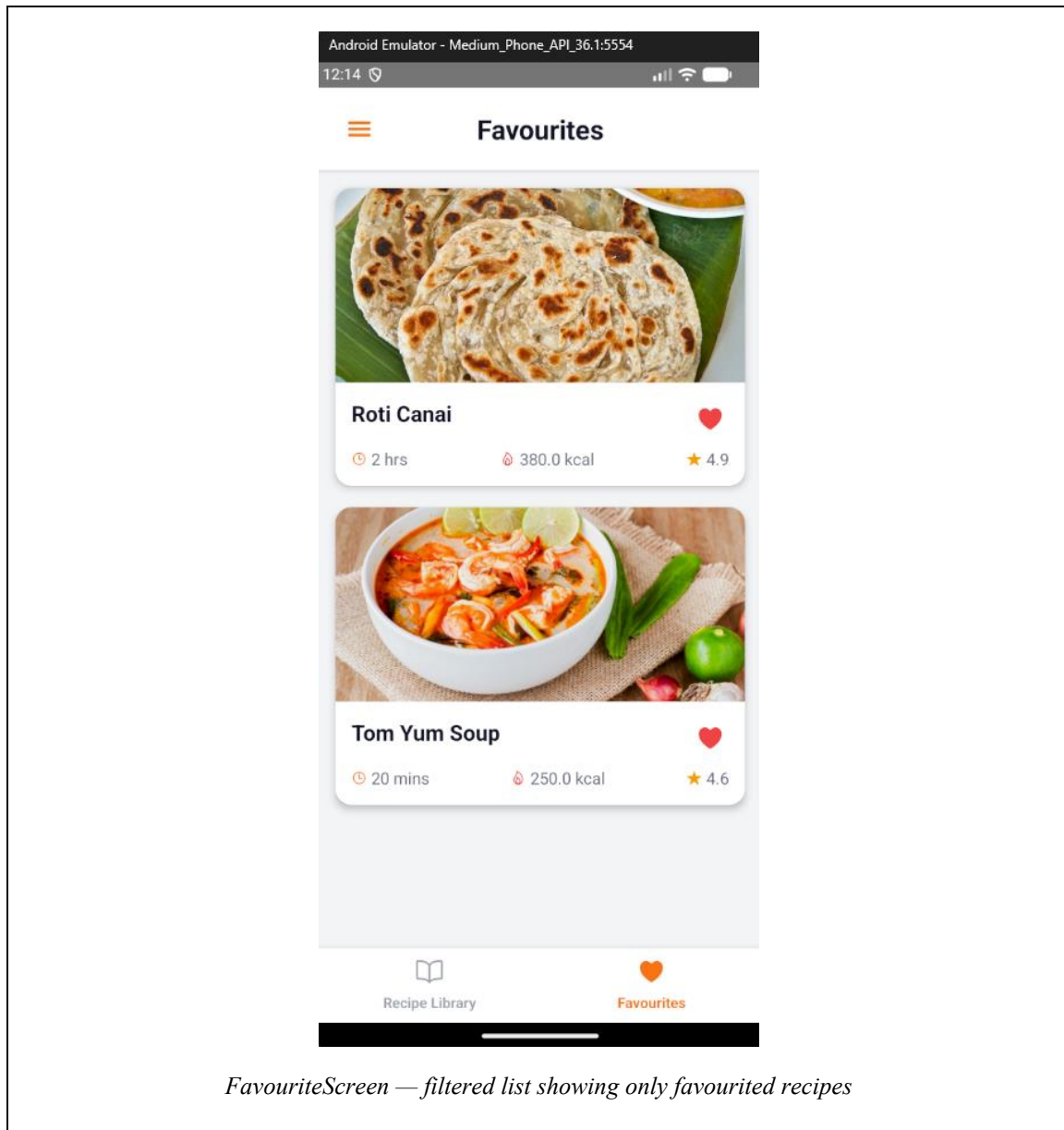


Figure A.2 – Favourites Screen

- **AddRecipeScreen:** A Stack screen accessed by tapping the add icon. It uses ScrollView to contain eight InputWithLabel form fields: Recipe Name (max 100 chars), Prep Time (max 20 chars), Image URL (max 500 chars), Rating 0-5, Description (multiline, max 1000 chars), Ingredients (multiline, max 2000 chars), Instructions (multiline, max 5000 chars), and Calories (numeric). The header has a close button on the left and a "CREATE" text link on the right.

Android Emulator - Medium_Phone_API_36.1:5554
12:15

× New Recipe CREATE

RECIPE NAME 0/100
e.g. Nasi Lemak

PREP TIME 0/20
e.g. 30 mins

IMAGE URL 0/500
https://example.com/image.jpg

RATING (0-5)
0.0 to 5.0

DESCRIPTION 0/1000
Fragrant coconut rice served with spicy sambal, anchovies, peanuts, and boiled egg

INGREDIENTS 0/2000
eg. Rice, Coconut Milk, Pandan, Anchovies, Peanuts, Chili, Egg

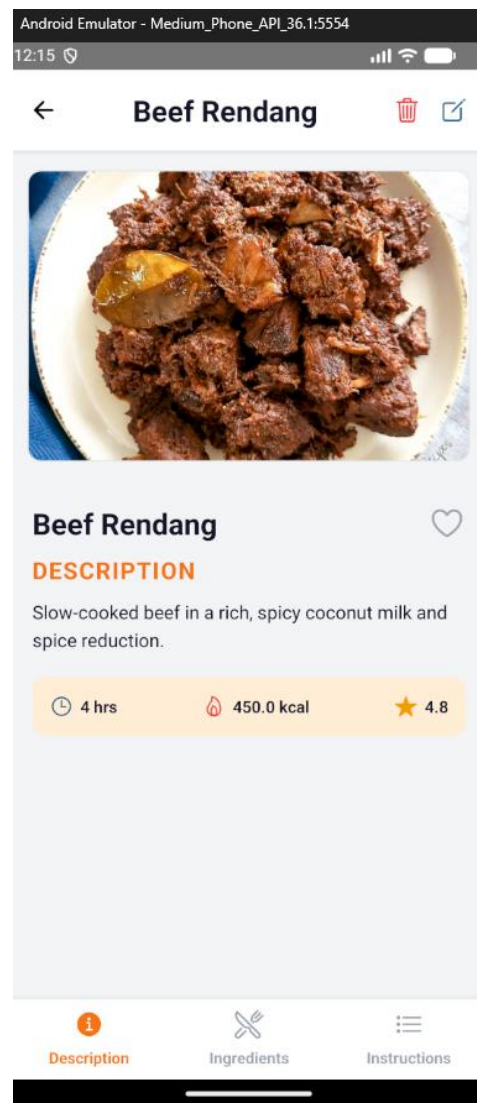
INSTRUCTIONS 0/5000
eg. 1. Cook rice with coconut milk and pandan.

CALORIES (KCAL)

AddRecipeScreen — scrollable form with all InputWithLabel fields and CREATE button

Figure A.3 – Add Recipe Screen

- **ViewRecipeScreen:** A Stack screen showing full recipe details. It displays a full-width Image banner or a placeholder View at the top, followed by a nested Bottom Tab Navigator with three content tabs: Description, Ingredients, and Instructions. The custom header includes a back arrow, the recipe title (truncated with numberOfLines=1), a trash icon, and an edit icon.



ViewRecipeScreen — full-width image banner and nested tab bar (Description tab)

Figure A.4 – View Recipe Screen (Description Tab)

- **EditRecipeScreen:** A Stack screen structurally mirroring AddRecipeScreen but pre-populating all form fields from the SQLite database. An ActivityIndicator spinner replaces the form while data is loading. The header has a close icon and a "SAVE" text button.

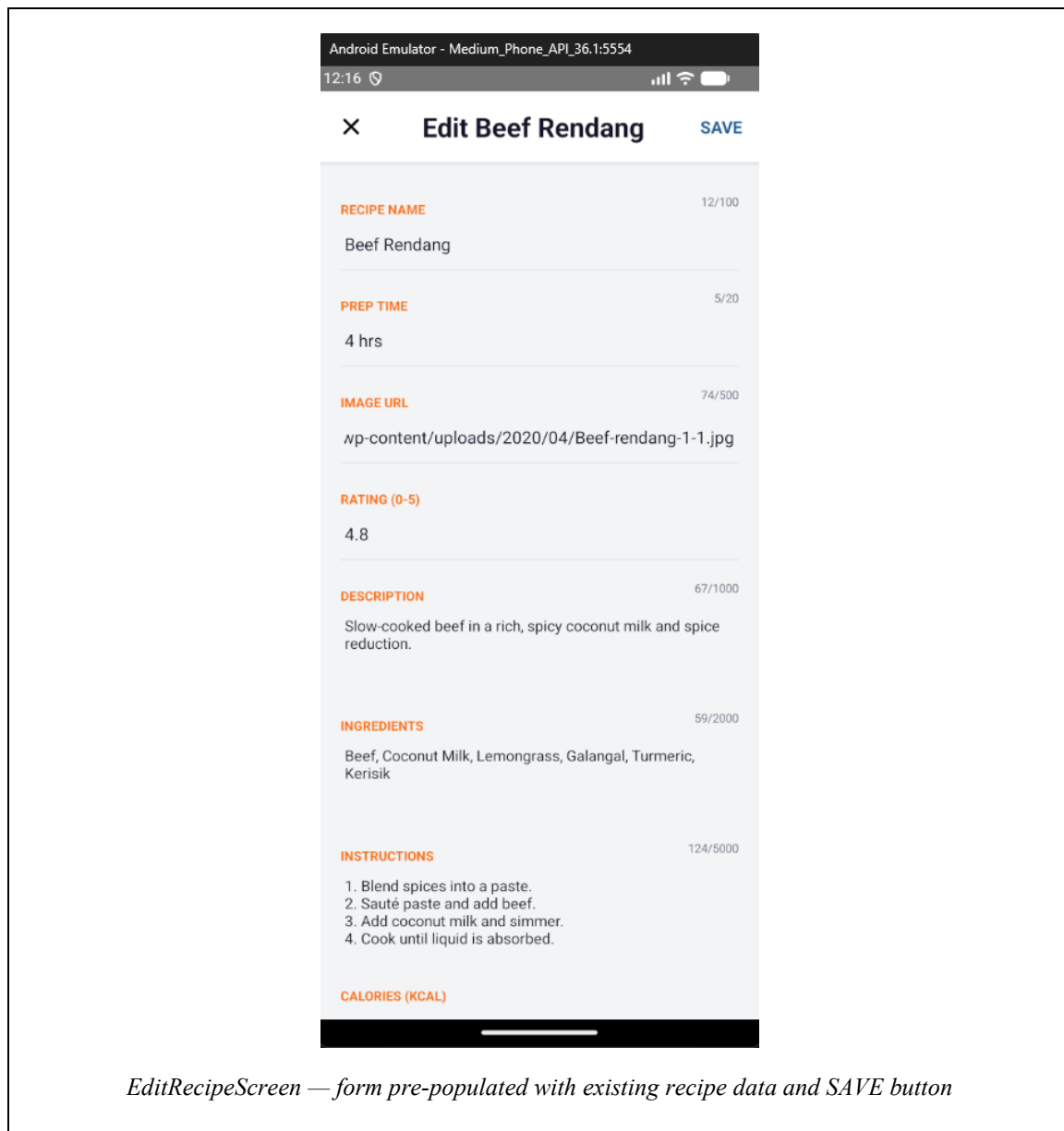


Figure A.5 – Edit Recipe Screen

- **CloudSyncScreen:** A Drawer screen using a centred syncContainer View layout. A status Text shows the WebSocket connection state. A syncCard View contains a cloud-outline Ionicons icon, the last backup timestamp, a divider line, and two AppButton components stacked vertically for backup and restore. ActivityIndicator replaces each button during active operations.

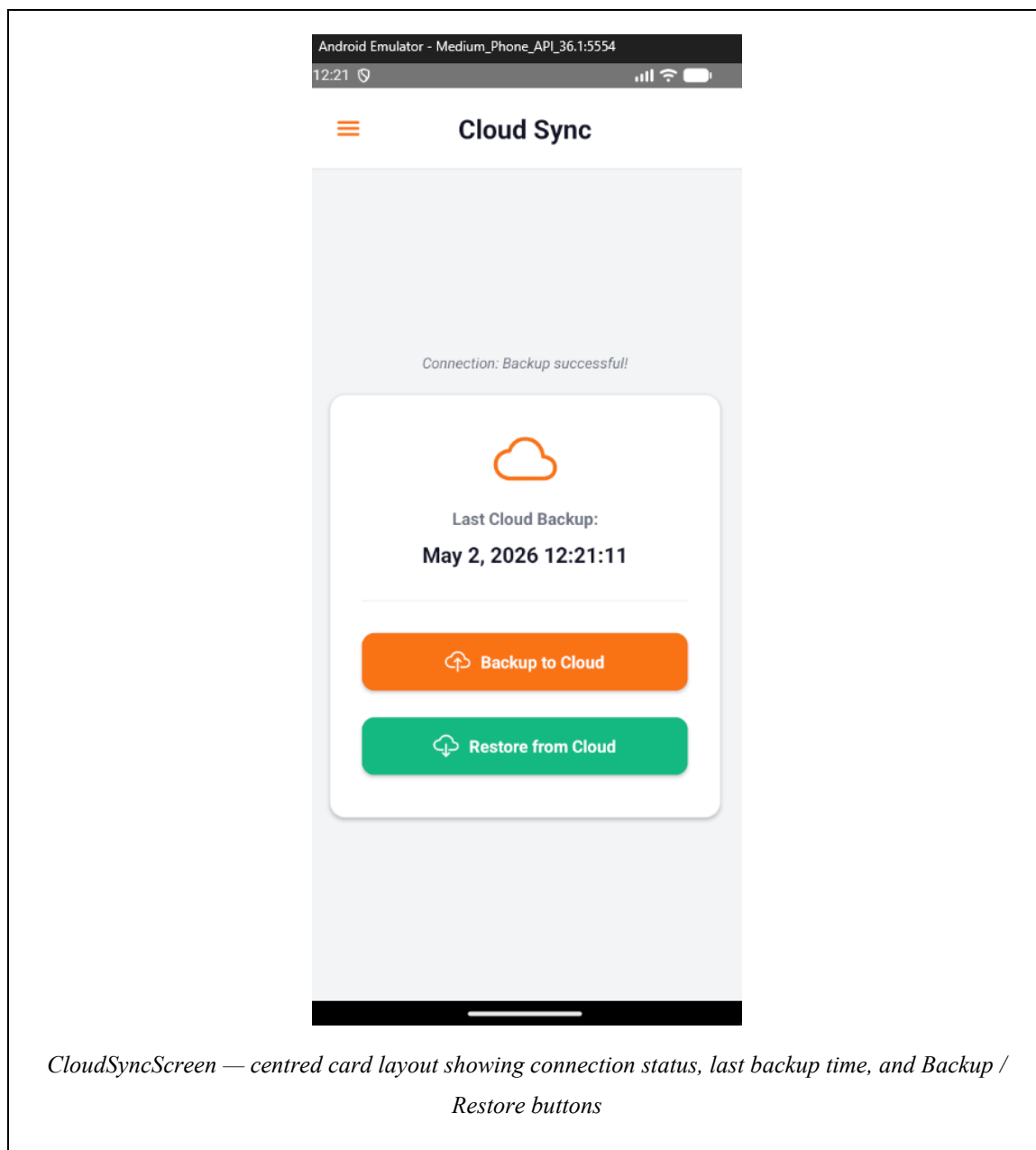


Figure A.6 – Cloud Sync Screen

- **FileBackupScreen:** A Drawer screen using a vertical layout. A syncCard container at the top holds the "Create New Backup File" AppButton. Below it, a section label shows the backup file count, followed by a FlatList of saved JSON files. Each file item card shows the filename (bold), creation date, and two side-by-side AppButtons, Import and Delete.

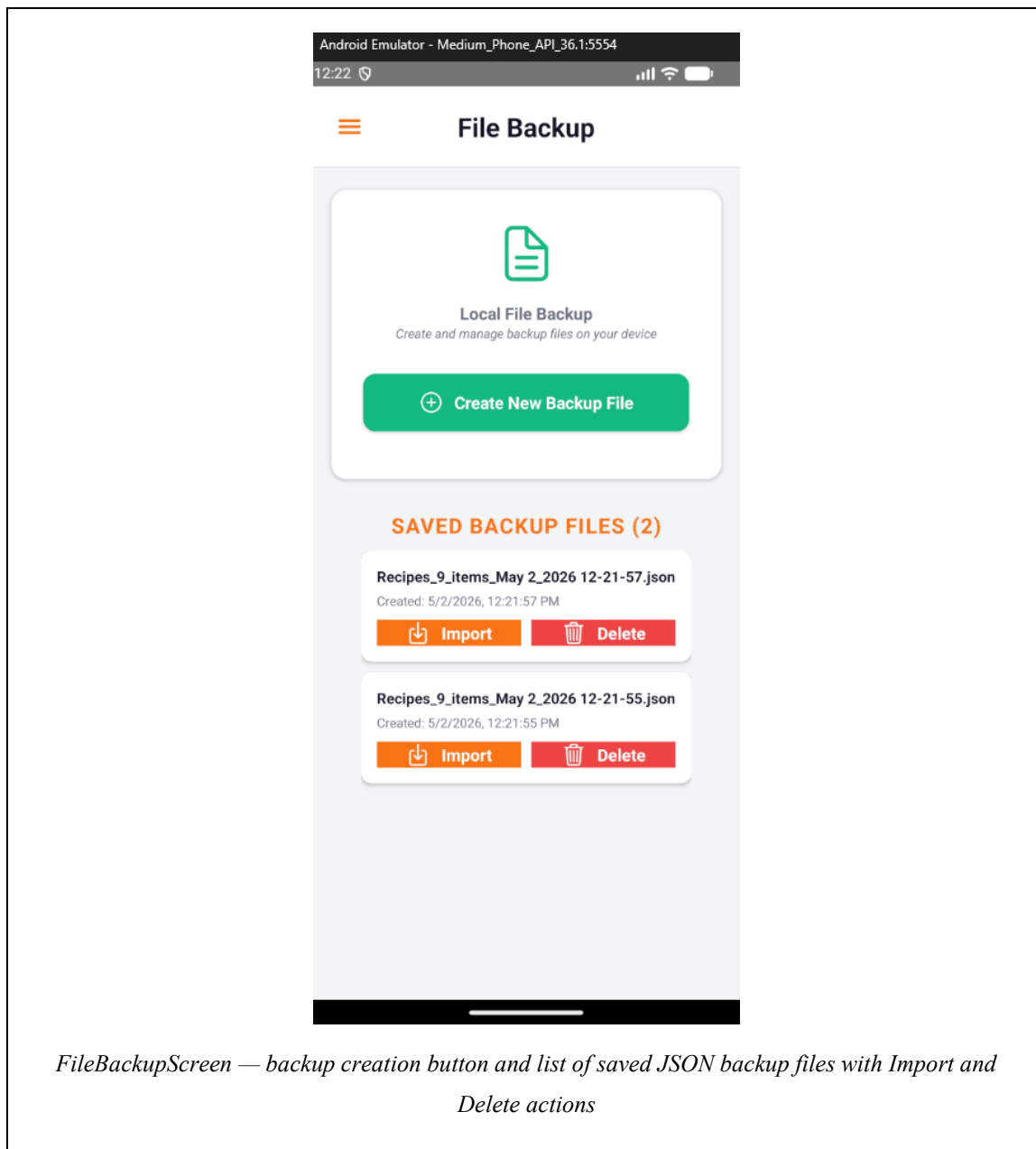


Figure A.7 – File Backup Screen

- **AboutScreen:** A Drawer screen using ScrollView. It shows a centred restaurant Ionicons logo, app name and version, and three labelled sections: ABOUT THIS APP, FEATURES (six FeatureItem rows), and DEVELOPER (credits text).

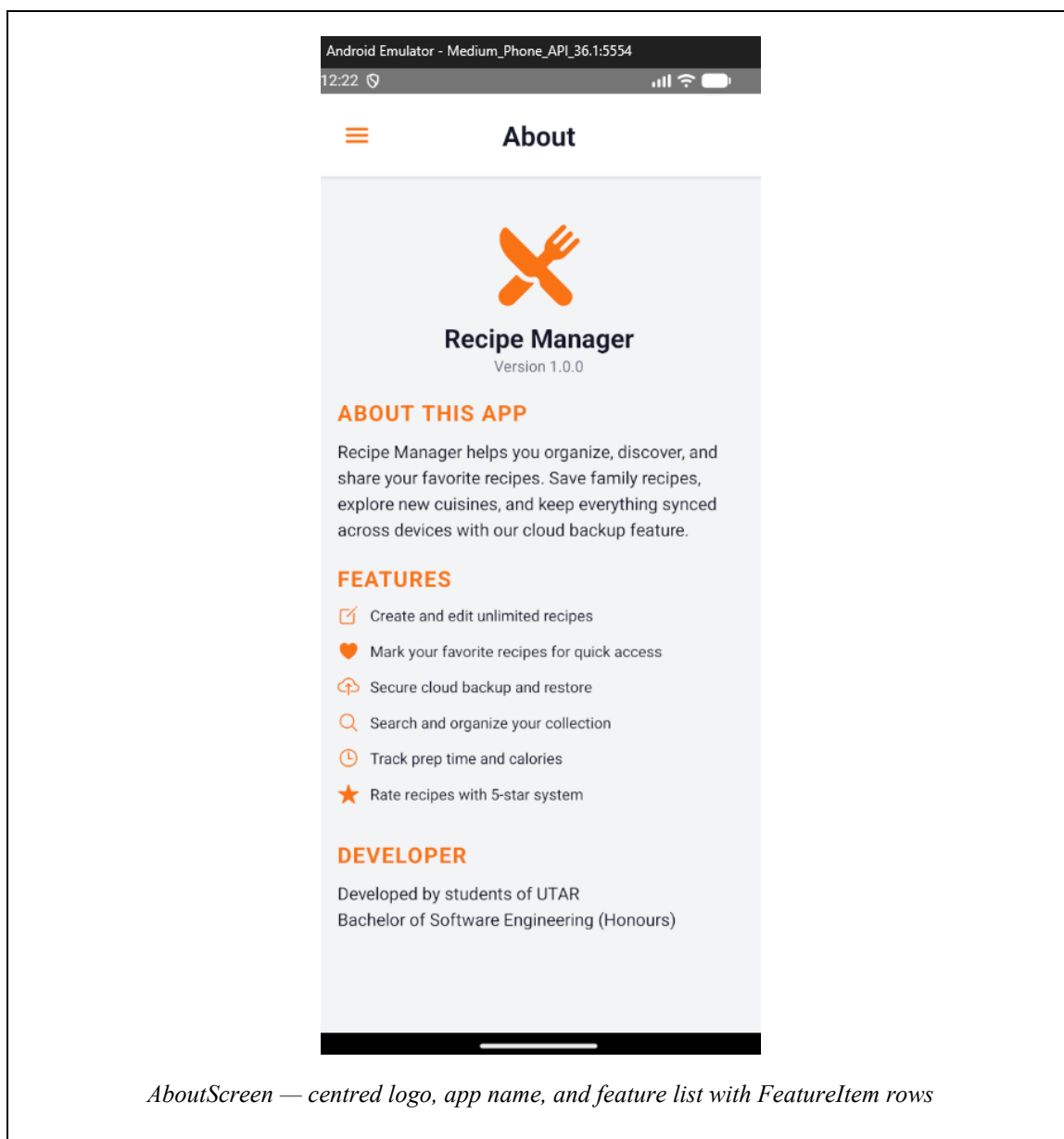


Figure A.8 – About Screen

All eight screens share the same colour palette and typography defined in `styles.tsx`, using the `Colours` object. The primary colour is #F97316 (orange) and the background is #F3F4F6 (light grey). This unified colour system ensures all screens feel part of the same cohesive application (React Native, 2026). The use of Ionicons from the `react-native-vector-icons` library provides consistent iconography across all screens, and the library supports scalable vector icons that render crisply at any resolution (Oblador, 2026).

A.2 Use of Components

The Recipe Manager App uses a combination of built-in React Native components and third-party libraries. As explained in UECS3253 Course Notes (2026), React Native provides a set of core built-in components that map directly to native platform views. The following components are used in the application:

- **View:** The fundamental container component used in every screen as the root layout element, as card wrappers in recipe list items (with elevation for shadow), as the header bar row, and as the metadata icon row in recipe cards. View supports all flexbox layout properties including flexDirection, justifyContent, alignItems, and flex.
- **Text:** Used for all visible text including screen titles, section labels, recipe titles, descriptions, ingredient lists, instruction steps, and empty-state messages. In ViewRecipeScreen, the recipe title uses numberOfLines=1 to truncate long names with an ellipsis.
- **Image:** Used in recipe cards and in ViewRecipeScreen's banner to display recipe photos from remote URLs via the image_url field in SQLite. The resizeMode='cover' prop fills the container without distortion. When image_url is null or empty, a View placeholder with an image-outline Icons icon is rendered instead.
- **ScrollView:** Used in AddRecipeScreen and EditRecipeScreen for the form scroll area, in the three detail tab screens inside ViewRecipeScreen, and in AboutScreen. Unlike FlatList, ScrollView renders all children at once and is suitable for screens with a small, finite number of elements such as form fields (React Native, no date).
- **FlatList:** Used in RecipeLibraryScreen, FavouriteScreen, and FileBackupScreen for scrollable lists that may contain many items. FlatList performs lazy rendering and supports the refreshing and onRefresh props for pull-to-refresh functionality.
- **TextInput:** Used in AddRecipeScreen and EditRecipeScreen for all recipe form fields. The multiline prop is applied to Description, Ingredients, and Instructions. The keyboardType='decimal-pad' prop is applied to Calories and Rating fields. The maxLength prop enforces character limits on each field.
- **TouchableOpacity:** Used for all icon buttons in headers, the hamburger menu button, the back arrow, the delete icon, and the edit icon. TouchableOpacity reduces the button's opacity on press to provide tactile visual feedback.

- **TouchableNativeFeedback:** Used inside the AppButton custom component to provide the Android ripple ink effect. This component is Android-specific and produces the material design ink ripple animation, which is the platform-standard touch feedback pattern on Android devices.
- **Alert:** Used for confirmation dialogs before all destructive or irreversible actions in AddRecipeScreen (confirm create), EditRecipeScreen (confirm save, discard), ViewRecipeScreen (confirm delete), CloudSyncScreen (confirm restore), and FileBackupScreen (confirm import and delete).
- **ToastAndroid:** Used for brief informational feedback messages that do not require user acknowledgement. It is used to confirm recipe creation, edit save, deletion, and favourite toggling. ToastAndroid.SHORT is used throughout, keeping notifications unobtrusive.
- **ActivityIndicator:** Used in EditRecipeScreen as a full-screen loading overlay while recipe data is being fetched from SQLite, and in CloudSyncScreen to dynamically replace Backup and Restore buttons during active network operations.
- **SafeAreaView:** Used in CloudSyncScreen as the root container to handle Android status bar and navigation bar insets, preventing content from being rendered under system UI elements.

The react-native-vector-icons library (Ionicons set) is used pervasively throughout the app for all iconographic elements. It provides scalable vector icons that render crisp at any size (Oblador, 2026). The react-native-sqlite-storage library provides SQLite database access. The @react-native-async-storage/async-storage library provides key-value storage for the favourites list (React Native, 2026A). The socket.io-client library handles WebSocket connections (Socket.IO, 2024). The react-native-fs library provides file system read and write access for local backup files.

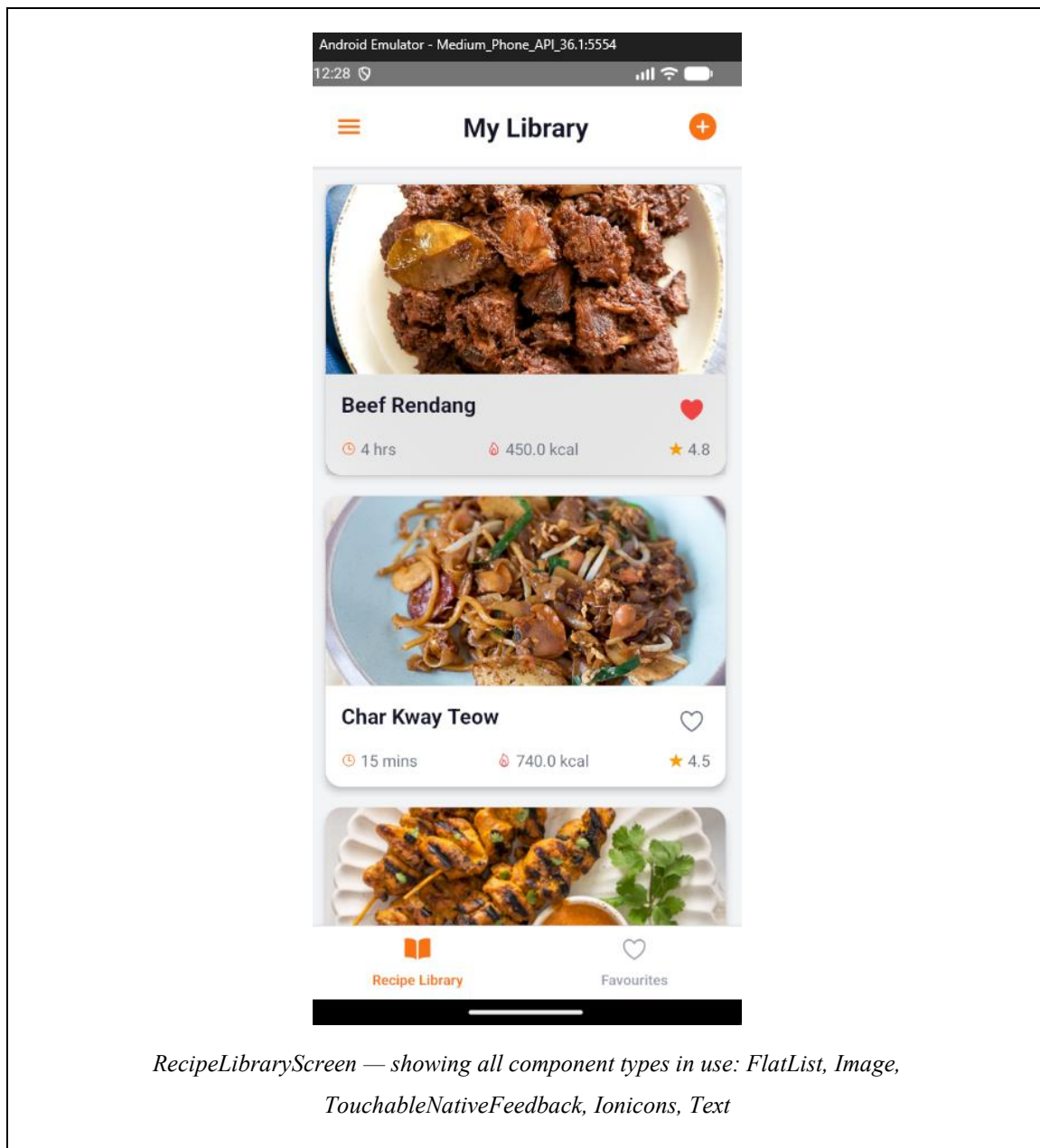


Figure A.9 – Component usage overview on RecipeLibraryScreen

A.3 Reusability (Custom Components)

Custom reusable components are defined in `UI.tsx` to promote code reuse and maintain a consistent visual style across the app. Creating custom components encapsulates repeated patterns of structure, styling, and behaviour into named abstractions. According to UECS3253 Course Notes (2026), a React functional component is a JavaScript function that returns a React element; all custom components in this app are functional components written in TypeScript. The practice of extracting repeated UI patterns into custom components directly supports React's principle of composability and single responsibility (React Native, 2026C).

InputWithLabel

`InputWithLabel` is a controlled form field component that wraps a `Text` label and a `TextInput` in a parent `View`. The component accepts: `label` (uppercase category label above the input), `value` (the controlled string value), `onChangeText` (callback for text changes), `maxLength` (integer character limit), `showCounter` (boolean to show a live character counter), `orientation` ('horizontal' for row layout or column by default), `style` (additional `TextInput` style override), and all remaining `TextInput` props via `...otherProps`. When `showCounter` is true and `maxLength` is provided, a secondary `Text` element displays the live character count to the right of the label in the format '24/100'. This single component handles all eight input fields in both `AddRecipeScreen` and `EditRecipeScreen`, replacing 16 instances of duplicated label-input markup with a single abstraction.

```
// UI.tsx - InputWithLabel custom component
export const InputWithLabel = (props: any) => {
  const { maxLength, value, label,
    showCounter = false, ...otherProps } = props;
  return (
    <View style={{ flexDirection: orientationDirection, marginBottom: 10 }}>
      <View style={{ flexDirection: 'row', justifyContent: 'space-between' }}>
        <Text style={styles.label}>{label}</Text>
        {showCounter && maxLength && (
          <Text>{value?.length || 0}/{maxLength}</Text>
        )}
      </View>
      <TextInput style={[styles.input, props.style]}
        placeholderTextColor="#9ca3af"
        maxLength={maxLength} value={value} {...otherProps}
      />
    </View>
  );
}
```

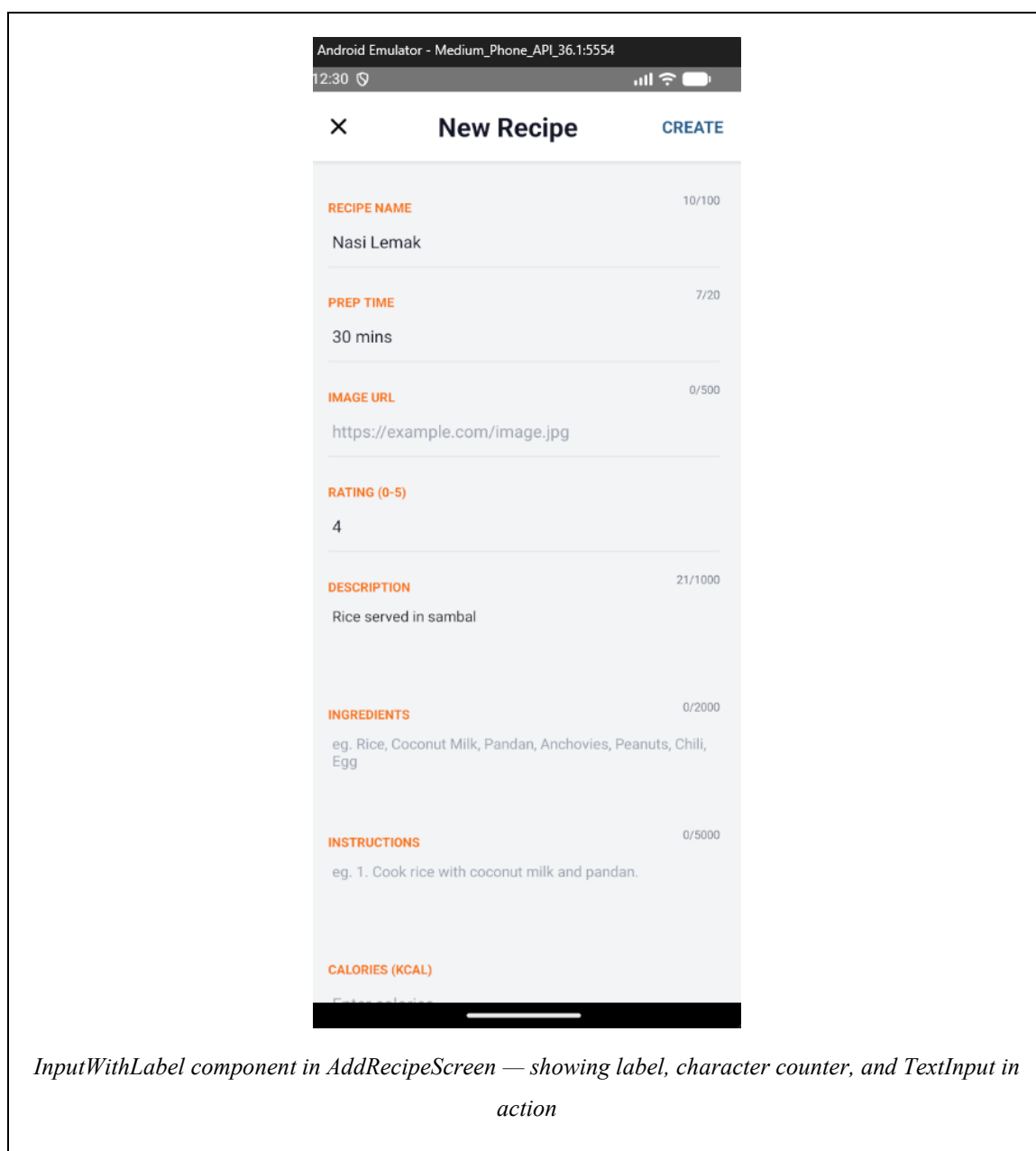


Figure A.10 – InputWithLabel custom component usage

AppButton

AppButton is a styled primary action button component that wraps a TouchableNativeFeedback (providing Android ripple feedback) around a View containing an optional Ionicons icon and a Text label. The component accepts: title (button label), icon (optional Ionicons icon name), onPress and onLongPress handlers, and theme ('primary' = orange #F97316, 'success' = green #10B981, 'danger' = red #EF4444). The three themes allow the same component to communicate different semantic meanings: orange for primary data

operations, green for constructive actions, and red for destructive actions. `AppButton` is used in `CloudSyncScreen` for two buttons and in `FileBackupScreen` for three types of buttons across multiple file cards.

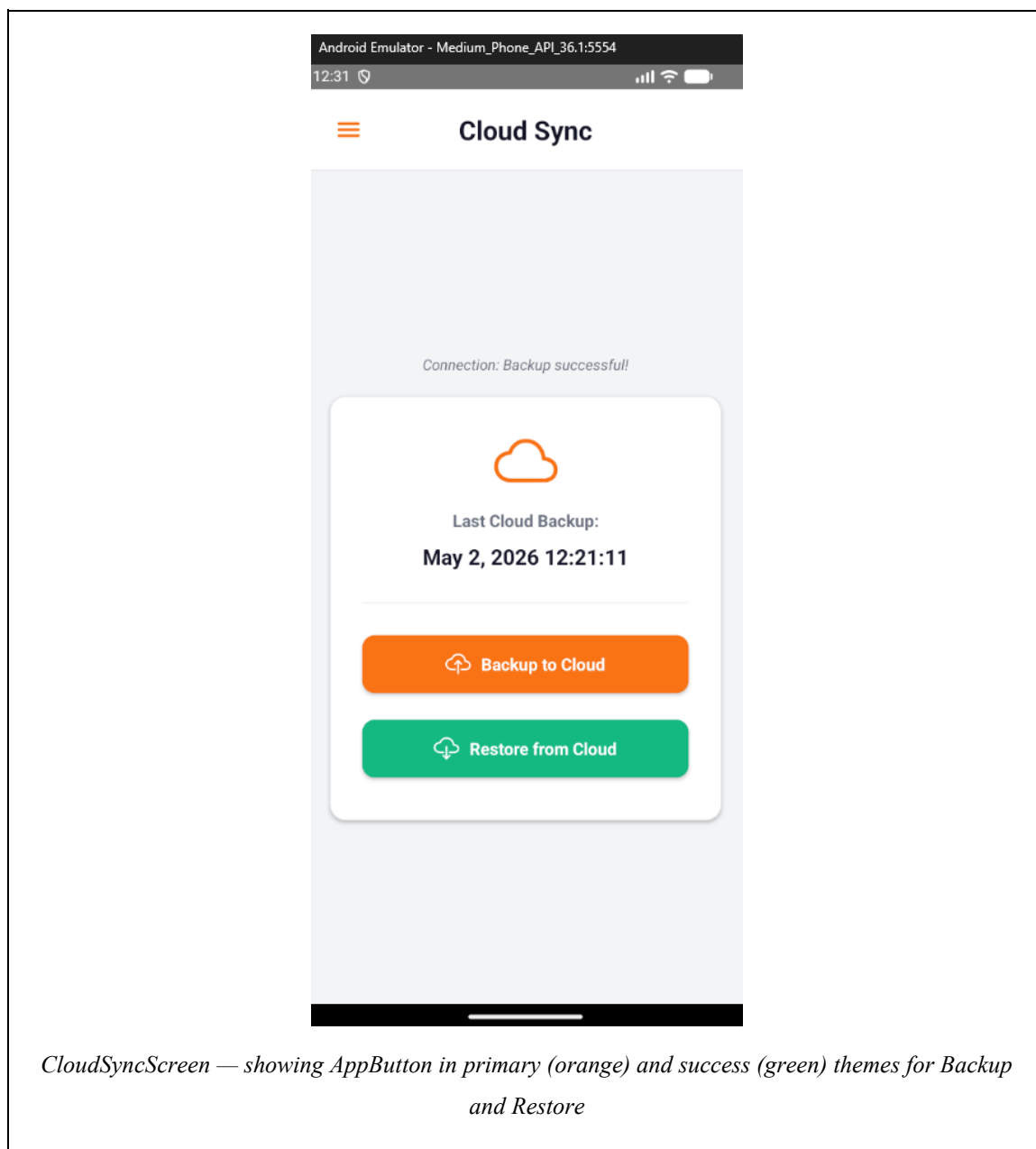


Figure A.11 – AppButton custom component themes

RecipeList

`RecipeList` is a `FlatList` wrapper that renders a scrollable list of recipe cards with pull-to-refresh support. It accepts: `data` (recipe objects array from SQLite with `is_favourite` boolean), `refreshing` (boolean for pull-to-refresh state), `onRefresh` (callback to trigger data reload), `emptyMessage` (string shown when data array is empty), `onItemPress` (callback receiving item

id for navigation), and onToggleFavourite (callback receiving the full item object). If data is null or empty, the component renders the emptyMessage in a styled empty-state Text paragraph. Each rendered card includes a View with borderRadius: 15 and elevation: 4, an Image (height 180) or placeholder View, a content View with a title row (bold recipe title and heart TouchableOpacity), and a detail row with three icon-text pairs for prep time, calories, and rating. This component is used identically in RecipeLibraryScreen and FavouriteScreen, removing all list-rendering code duplication between the two-tab screens.

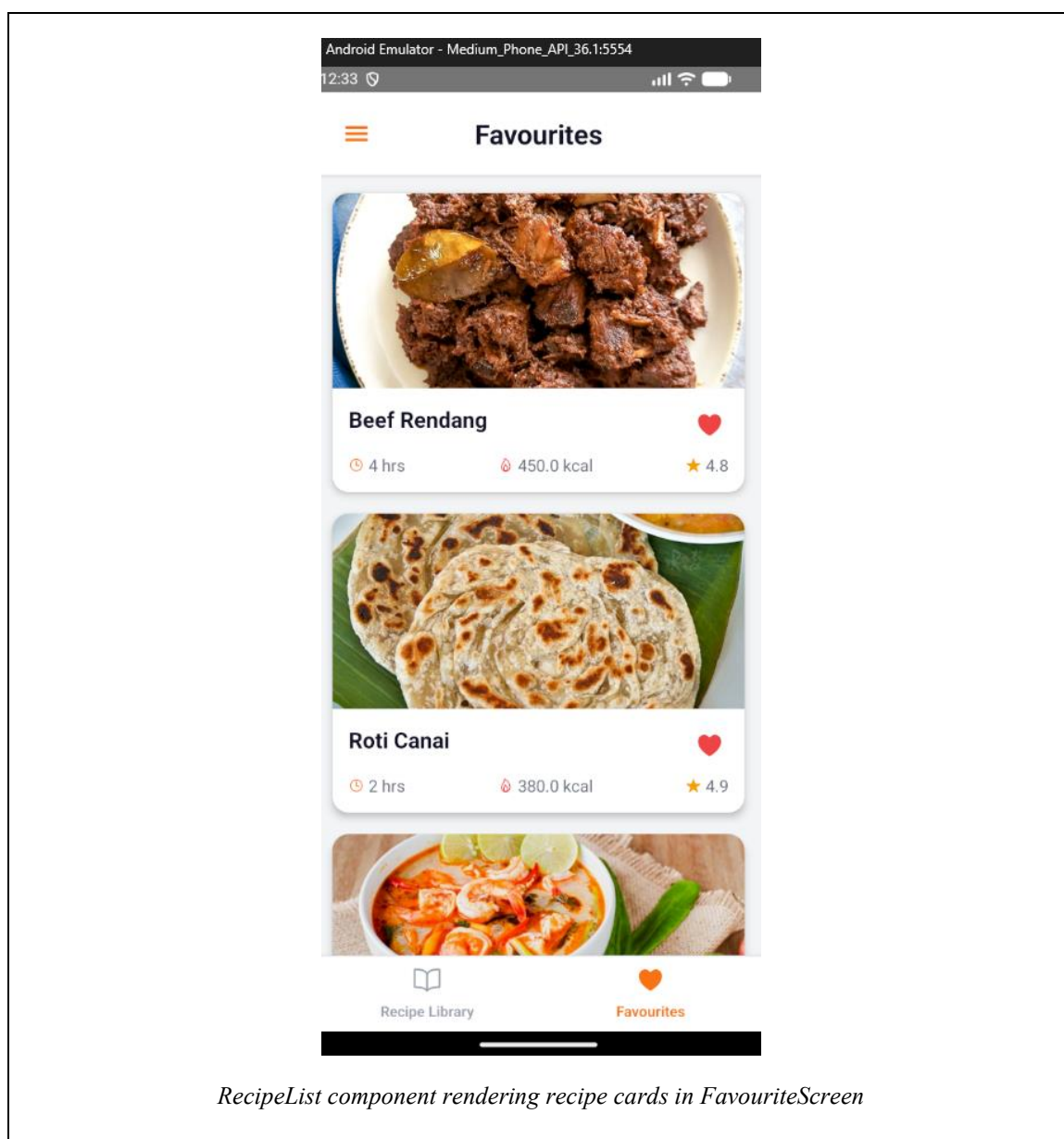


Figure A.12 – RecipeList custom component rendering

FeatureItem

FeatureItem is a small presentational component used in AboutScreen. It accepts icon (Ionicons icon name) and text (description string) props. It renders a horizontal row with the Ionicons icon on the left and the description text on the right, with a bottom margin of 12. This component is used six times in AboutScreen's FEATURES section.

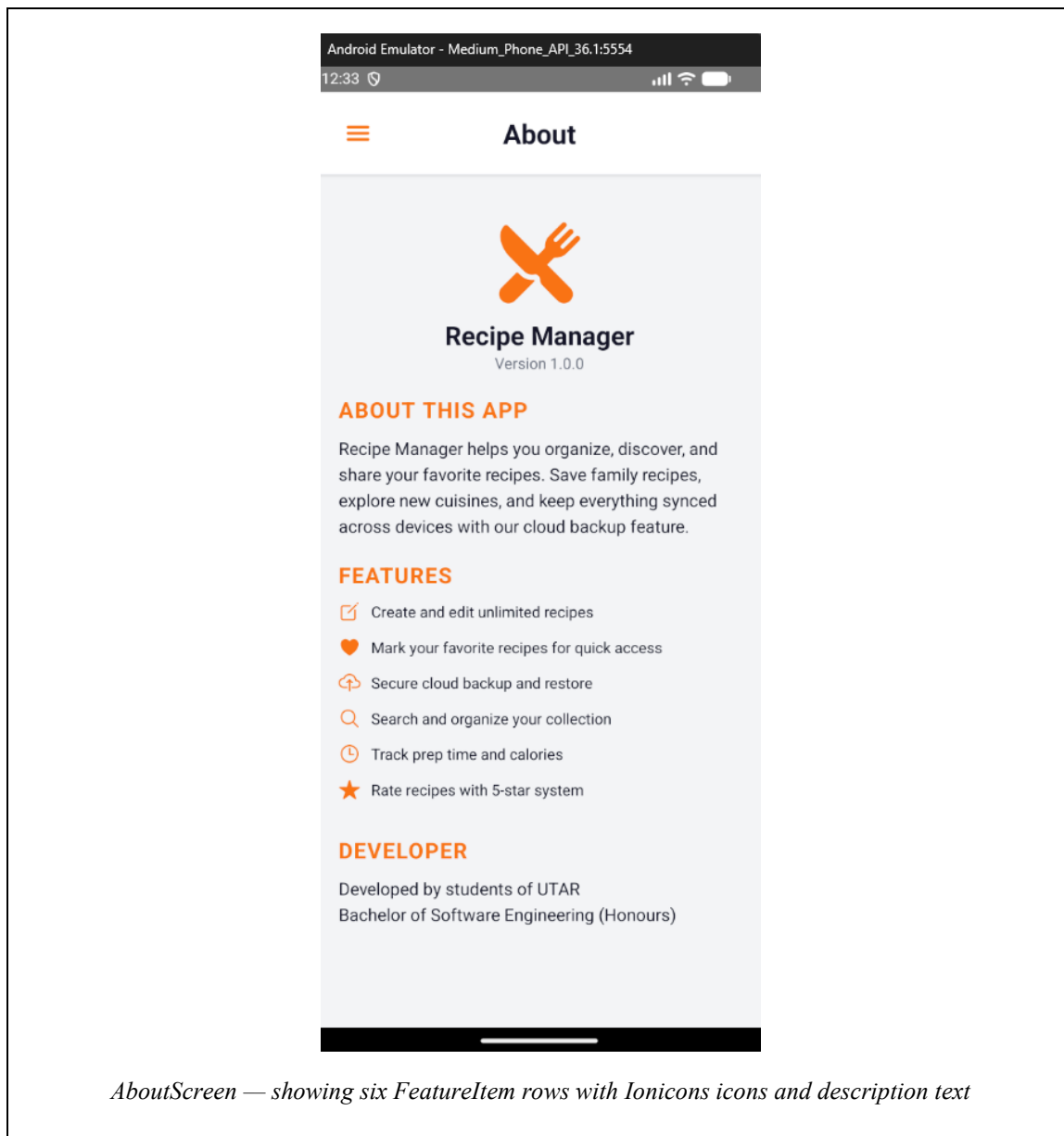


Figure A.13 – FeatureItem custom component in AboutScreen

A.4 Implementation of React-Navigation

React Navigation is a JavaScript-based navigation library for React Native that implements Stack, Drawer, and Tab navigator patterns with consistent behaviour on both Android and iOS (Wieruch, 2023). According to React Navigation (2026B), it is built entirely in JavaScript and does not depend on native navigation APIs, making it fully customisable and platform-agnostic. React Navigation is the recommended navigation solution because it offers the greatest configurability and flexibility and integrates well with React's component model (UECS3253 Course Notes, 2026). The Recipe Manager App implements all three navigator types in a nested hierarchy defined in App.tsx, with TypeScript generic type parameters for route parameter type safety.

Stack Navigator

The Stack Navigator is the root-level navigator of the entire application, created using `createStackNavigator<RootStackParams>()` from `@react-navigation/stack` and wrapped inside a `NavigationContainer`. The Stack Navigator manages screen transitions by pushing new screens on top of a stack; calling `navigation.goBack()` pops the top screen and returns to the previous one. The `RootStackParams` type defines four routes: `DrawerRoot` (the root drawer navigator rendered as the initial screen), `AddRecipe` (`AddRecipeScreen`, navigated to from `RecipeLibraryScreen` with a refresh callback), `ViewRecipe` (`ViewRecipeScreen`, navigated to from recipe card taps with `id` and refresh callback), and `EditRecipe` (`EditRecipeScreen`, navigated to from `ViewRecipeScreen` with `id`, `refresh`, and `homeRefresh` callbacks). All four routes have `headerShown: false` to allow each screen to manage its own custom header.

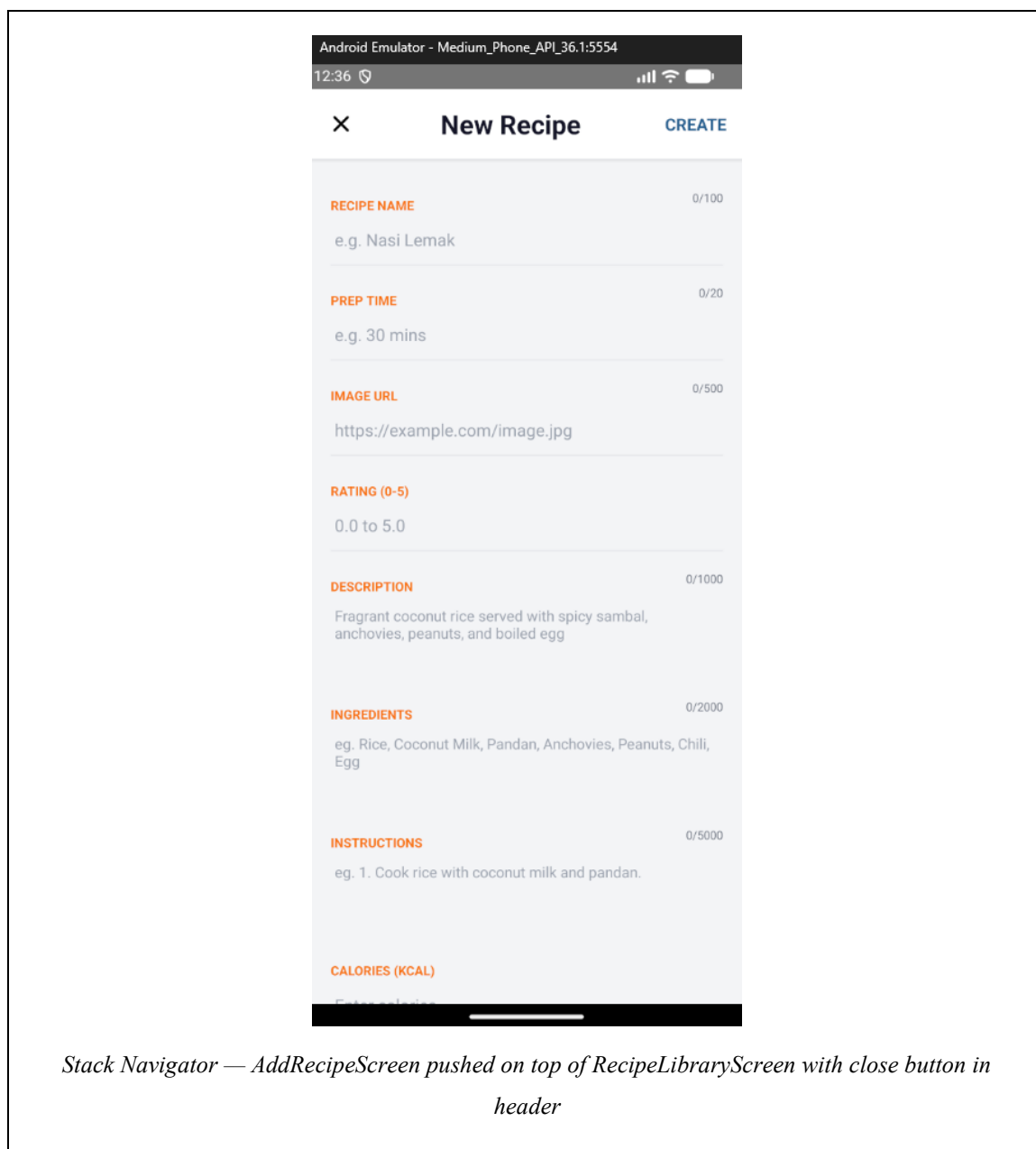
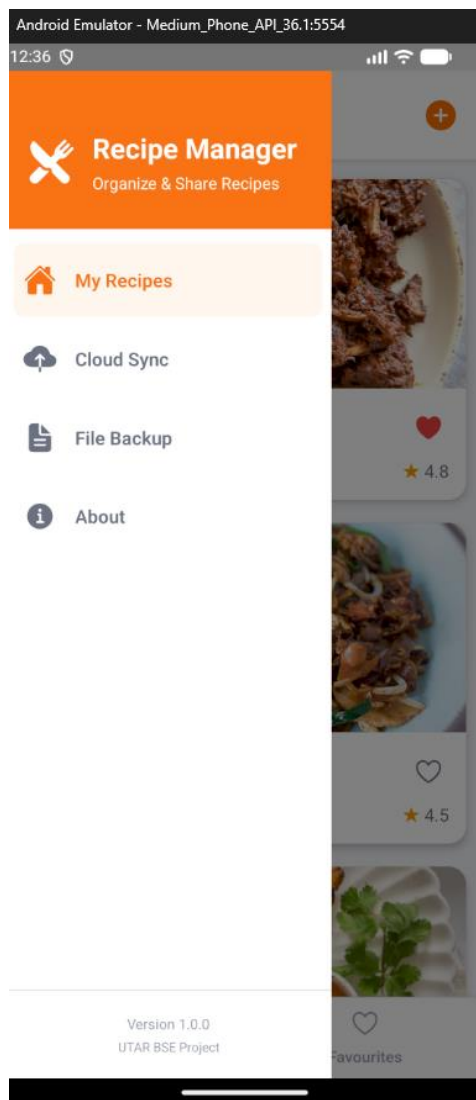


Figure A.14 – Stack Navigator (AddRecipeScreen pushed)

Drawer Navigator

The Drawer Navigator is created using `createDrawerNavigator<DrawerParams>()` and contains four screens accessible via a side-slide drawer panel. A custom drawer component (`CustomDrawerContent`) renders a branded header section at the top with the app logo and app name in a full-width View, followed by the standard `DrawerItemList` and a version footer. The `DrawerParams` type defines four routes: Home (`MainTabs`, the main tab navigator), Cloud Sync (`CloudSyncScreen`), File Backup (`FileBackupScreen`), and About (`AboutScreen`). The drawer is styled with `drawerActiveTintColor`: orange for the selected item, `drawerInactiveTintColor`: grey for inactive items, and consistent padding for each drawer item.



Drawer Navigator open state — showing custom header with app logo, My Recipes, Cloud Sync, File Backup, and About items

Figure A.15 – Drawer Navigator (custom drawer content)

Bottom Tab Navigator

The Bottom Tab Navigator is created using `createBottomTabNavigator<TabParams>()` and renders two tabs: Recipe Library (RecipeLibraryScreen) and Favourites (FavouriteScreen). The tab bar is configured with `tabBarActiveTintColor`: orange, `tabBarInactiveTintColor`: light gray, `height`: 70, and `paddingBottom`: 10. Tab icons switch between filled and outline Ionicons variants based on the focused state: book / book-outline for Recipe Library, and heart / heart-outline for Favourites. Both tab screens use the `useIsFocused()` hook to detect when they become active, triggering an automatic data reload each time the user switches to them. A

second nested Bottom Tab Navigator is used inside ViewRecipeScreen to present three content panels: Description, Ingredients, and Instructions.

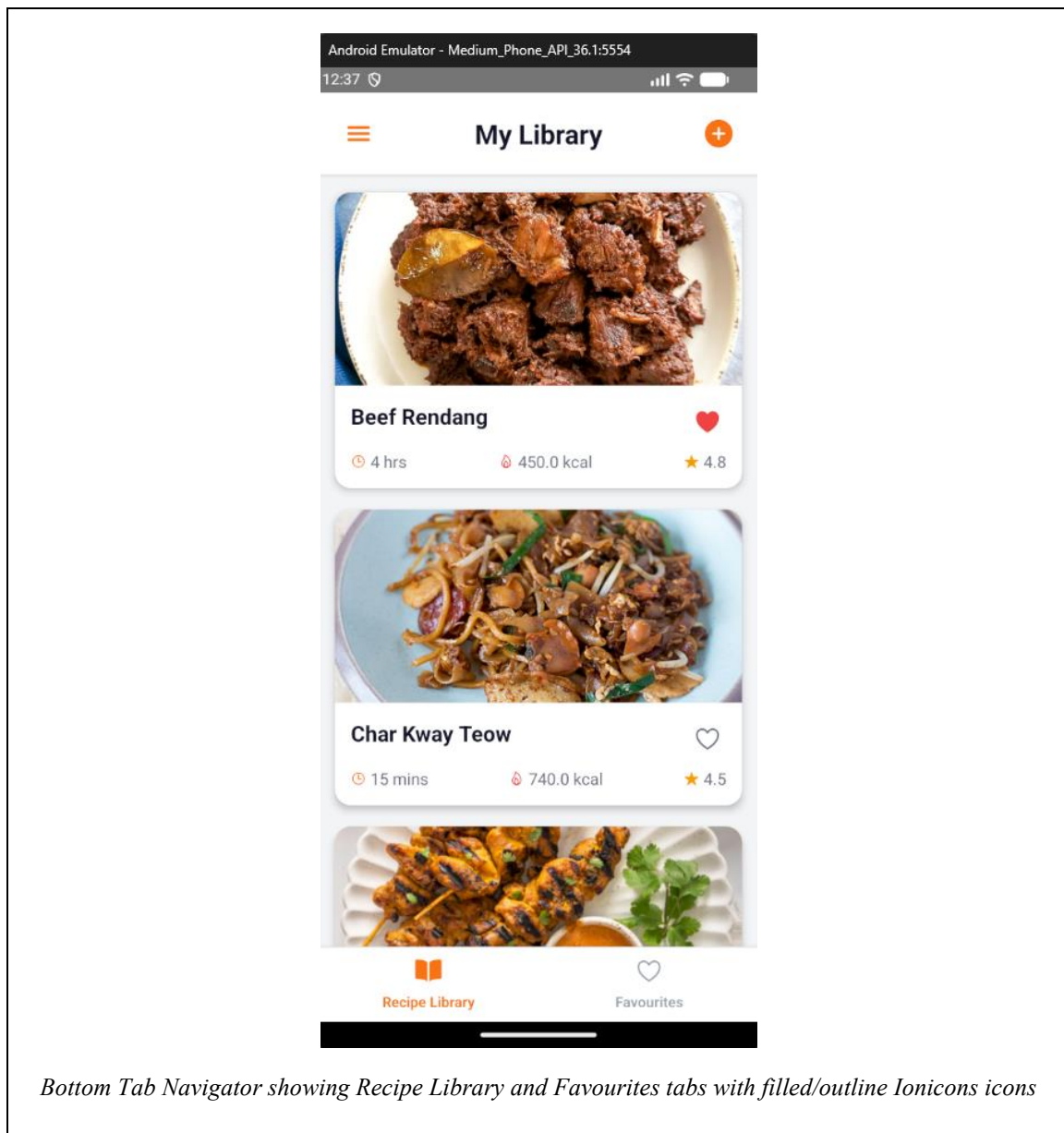
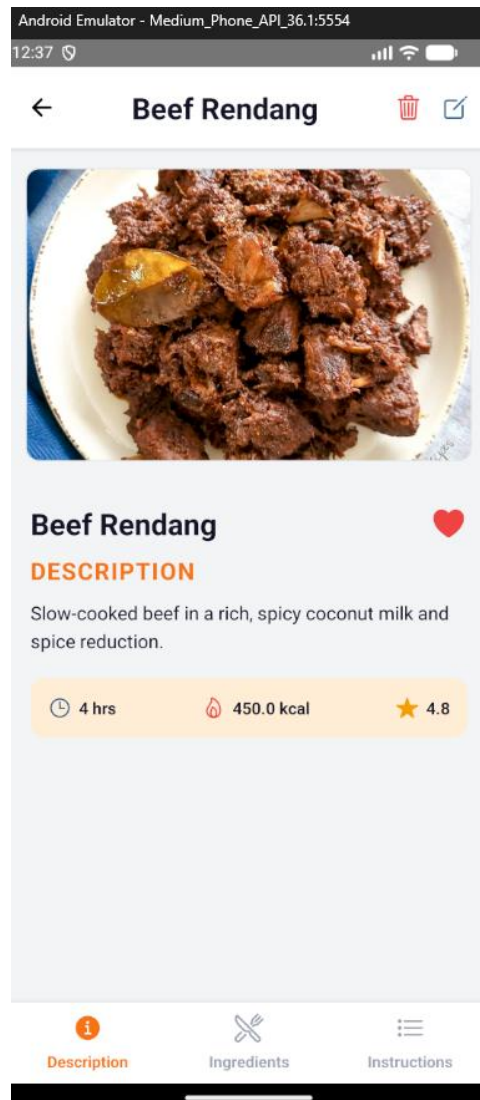


Figure A.16 – Bottom Tab Navigator



Nested Bottom Tab Navigator inside ViewRecipeScreen — Description, Ingredients, Instructions tabs

Figure A.17 – Nested Tab Navigator in ViewRecipeScreen

B DATA PERSISTENCE

B.1 Usage of Data Persistence

Data persistence is a critical requirement for the Recipe Manager App. All user-created and modified recipe records must survive application termination and device restarts, the favourites list must persist independently across sessions, and users must be able to create restorable backup copies of their data. To address these requirements, the app uses two built-in persistence mechanisms, SQLite and AsyncStorage, and one user-controlled file-system backup mechanism. As stated in UECS3253 Course Notes (2026), choosing the appropriate persistence strategy depends on the nature and structure of the data being stored.

SQLite is used as the primary persistence layer for recipe data. SQLite is a self-contained, serverless, zero-configuration, transactional SQL database engine embedded directly within the application process (SQLite Consortium, 2025). According to UECS3253 Course Notes (2026), SQLite is well-suited for structured relational data in mobile applications because it supports complex SQL queries, enforces data types and constraints, handles ACID transactions, and requires no network connectivity. The react-native-sqlite-storage library provides Promise-based JavaScript bindings for React Native. The application creates a single SQLite database file named `recipes.sqlite` containing one table, `recipes` with columns: `id` (INTEGER PRIMARY KEY AUTOINCREMENT), `title` (VARCHAR(100)), `description` (TEXT), `instructions` (TEXT), `ingredients` (TEXT), `prep_time` (VARCHAR(50)), `image_url` (TEXT), `calories` (DOUBLE), and `rating` (REAL DEFAULT 0).

AsyncStorage is used as the secondary persistence layer for the favourites list. AsyncStorage is an unencrypted, asynchronous, persistent key-value storage system that is global to the app (React Native, 2026A). The `@react-native-async-storage/async-storage` package provides the implementation. In this app, the entire favourites list is stored as a single JSON-serialised array of integer recipe IDs under the key `'@favourite_recipes'`. The decision to store favourites in AsyncStorage rather than as a boolean column in the SQLite `recipes` table is deliberate: toggling a favourite requires only an AsyncStorage read-modify-write operation rather than a full SQL UPDATE, which is significantly cheaper, and the favourites state can be managed and backed up independently of the recipe schema.

The react-native-fs (RNFS) library provides file system access for the local file backup feature. Users can manually export a JSON snapshot of all SQLite recipe data and AsyncStorage favourites data to a file in the device's document directory. This file persists indefinitely until

the user deletes it. This mechanism is not automatic, it requires deliberate user action from the File Backup screen, making it a complementary safety layer rather than a background persistence mechanism. All three persistence layers are combined during the cloud backup and restore workflow: the `_exportDataToJson` function reads from both SQLite and AsyncStorage to produce a single comprehensive JSON snapshot.

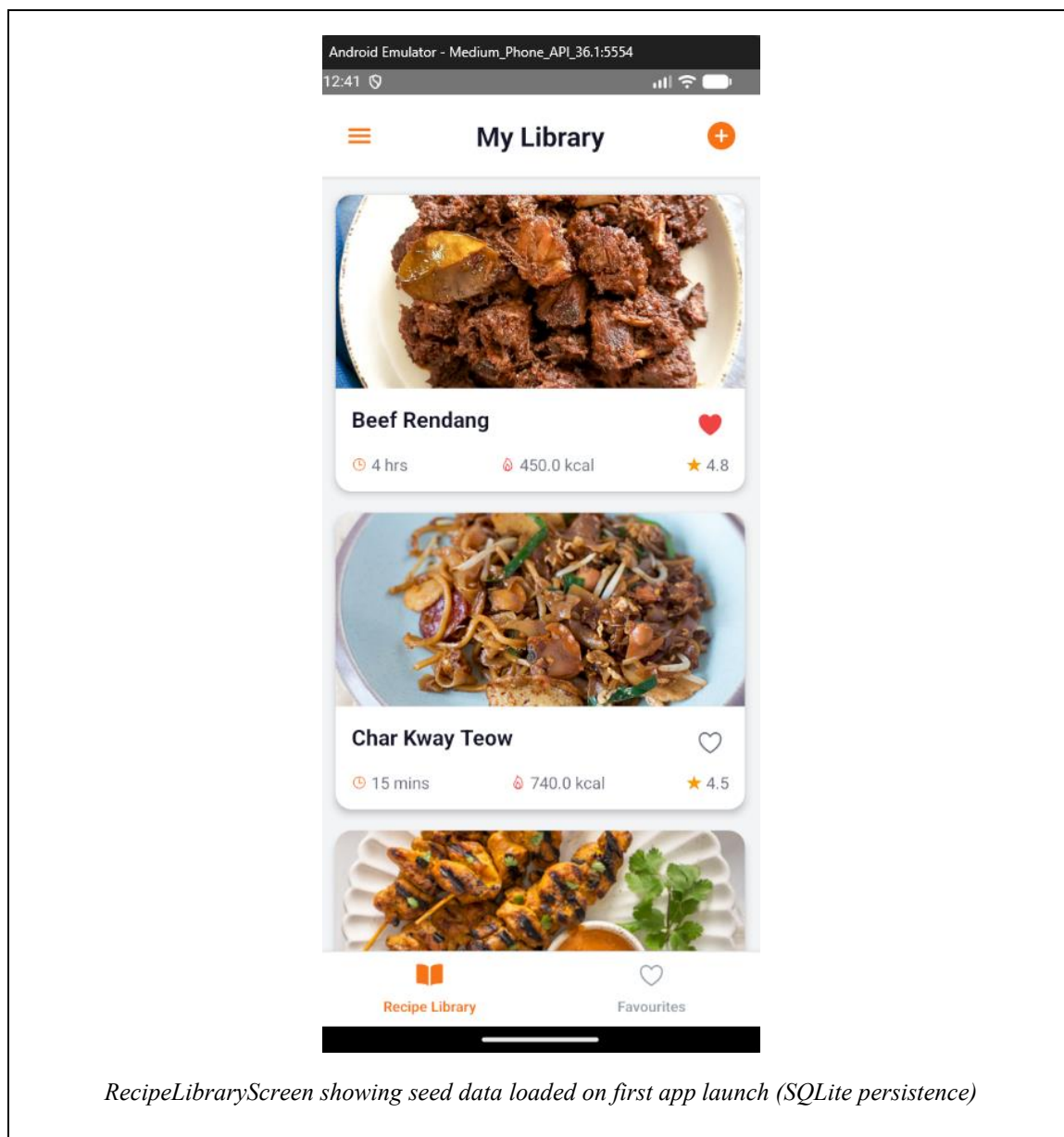


Figure B.0 – SQLite seed data on first launch

B.2 Implementation of Data Persistence

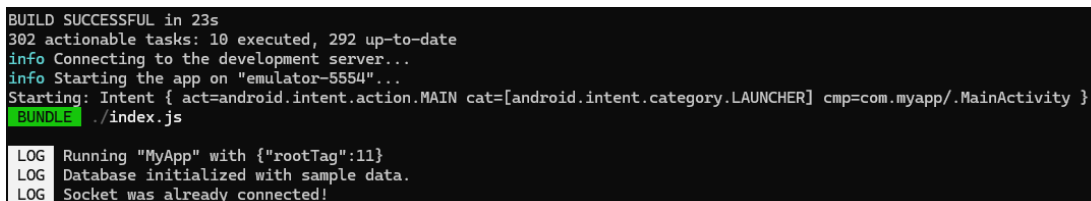
The persistence implementation is organised into two service modules. The `db-service.ts` module contains all SQLite database operations and file system operations. The `favourite-service.ts` module contains all AsyncStorage operations for the favourites list. This separation

follows the service layer pattern, keeping data access logic cleanly isolated from the UI rendering logic in screen components. All database operations begin by calling the PromiseDB helper function, which wraps the SQLite.openDatabase call in a native Promise, converting the callback-based API to async/await syntax consistent with the approach demonstrated in UECS3253 Course Notes (2026).

Database Initialisation — `_createTables`

`_createTables` is called once at application startup from the `useEffect` hook in `App.tsx`. It calls PromiseDB to obtain a database handle, then executes `CREATE TABLE IF NOT EXISTS recipes(...)` to create the table schema. It then performs a `SELECT COUNT(*)` check; if the result is zero (first launch), it inserts nine sample recipes covering Malaysian classics (Nasi Lemak, Beef Rendang, Chicken Satay, Char Kway Teow, Laksa Nyonya, Roti Canai) and international dishes (Margherita Pizza, Pad Thai, Tom Yum Soup) as seed data.

```
// db-service.ts - _createTables()
export const _createTables = async () => {
  const db: any = await PromiseDB();
  await db.executeSql(`CREATE TABLE IF NOT EXISTS recipes(
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title VARCHAR(100), description TEXT,
    instructions TEXT, ingredients TEXT,
    prep_time VARCHAR(50), image_url TEXT,
    calories DOUBLE, rating REAL DEFAULT 0
  )`);
  db.executeSql('SELECT COUNT(*) as count FROM recipes', [],
    (results: any) => {
      if (results.rows.item(0).count === 0) {
        // INSERT seed data for 9 sample recipes
      }
    }
  );
};
```



```
BUILD SUCCESSFUL in 23s
302 actionable tasks: 10 executed, 292 up-to-date
info Connecting to the development server...
info Starting the app on "emulator-5554"...
Starting: Intent { act=android.intent.action.MAIN cat=[android.intent.category.LAUNCHER] cmp=com.myapplication.MainActivity }
BUNDLE ./index.js
LOG Running "MyApp" with {"rootTag":11}
LOG Database initialized with sample data.
LOG Socket was already connected!
```

App.tsx showing `_createTables` called in `useEffect`, and `RecipeLibraryScreen` with seed data loaded

Figure B.1 – SQLite table initialisation and seed data (`db-service.ts`)

Query Functions

The `_queryRecipes` function fetches all recipes for the `RecipeLibraryScreen`. It first calls `_getFavourites` from `favourite-service.ts` to retrieve the `AsyncStorage` favourites array, then executes `SELECT * FROM recipes ORDER BY title` to retrieve all records sorted alphabetically. The results from `results.rows.raw()` are passed through `Array.map` to attach an `is_favourite`: boolean field by checking whether each recipe's id is included in the `favIds` array. This cross-layer data join is performed in JavaScript after the SQL query, keeping the SQL simple while still enriching the result objects with the correct favourite state.

The `_queryFavouriteRecipes` function is used by `FavouriteScreen`. It first checks if the `favIds` array is empty and immediately returns `[]` if so, avoiding an unnecessary database query. Otherwise, it builds a dynamic SQL IN clause: `SELECT * FROM recipes WHERE id IN (?, ?, ?)` `ORDER BY title`, with one `?` placeholder per favourite ID. All favourite IDs are passed as parameterised arguments, preventing SQL injection. The `_queryRecipeById` function retrieves a single record using `SELECT * FROM recipes WHERE id = ?` and returns `results.rows.raw()[0]`, the first row directly, which is used by `ViewRecipeScreen` and `EditRecipeScreen` to load a recipe's full details after navigation.

AsyncStorage — favourite-service.ts

The `_getFavourites` function reads the '@favourite_recipes' key using `AsyncStorage.getItem`. If the key exists, `JSON.parse` converts the stored string back to a number array; if the key is null (first app launch), an empty array is returned as the default. The `_toggleFavouriteAsync` function reads the current array, uses `Array.filter` to remove the id if already present, or `Array.push` to add it if not, then writes the updated array back with `AsyncStorage.setItem`. It returns the new boolean favourite state, which the screen component uses to update the heart icon and show a toast message. The `_checkIsFavourite` function simply calls `_getFavourites` and tests `Array.includes` for the given id.

```
// favourite-service.ts
const FAV_KEY = '@favourite_recipes';

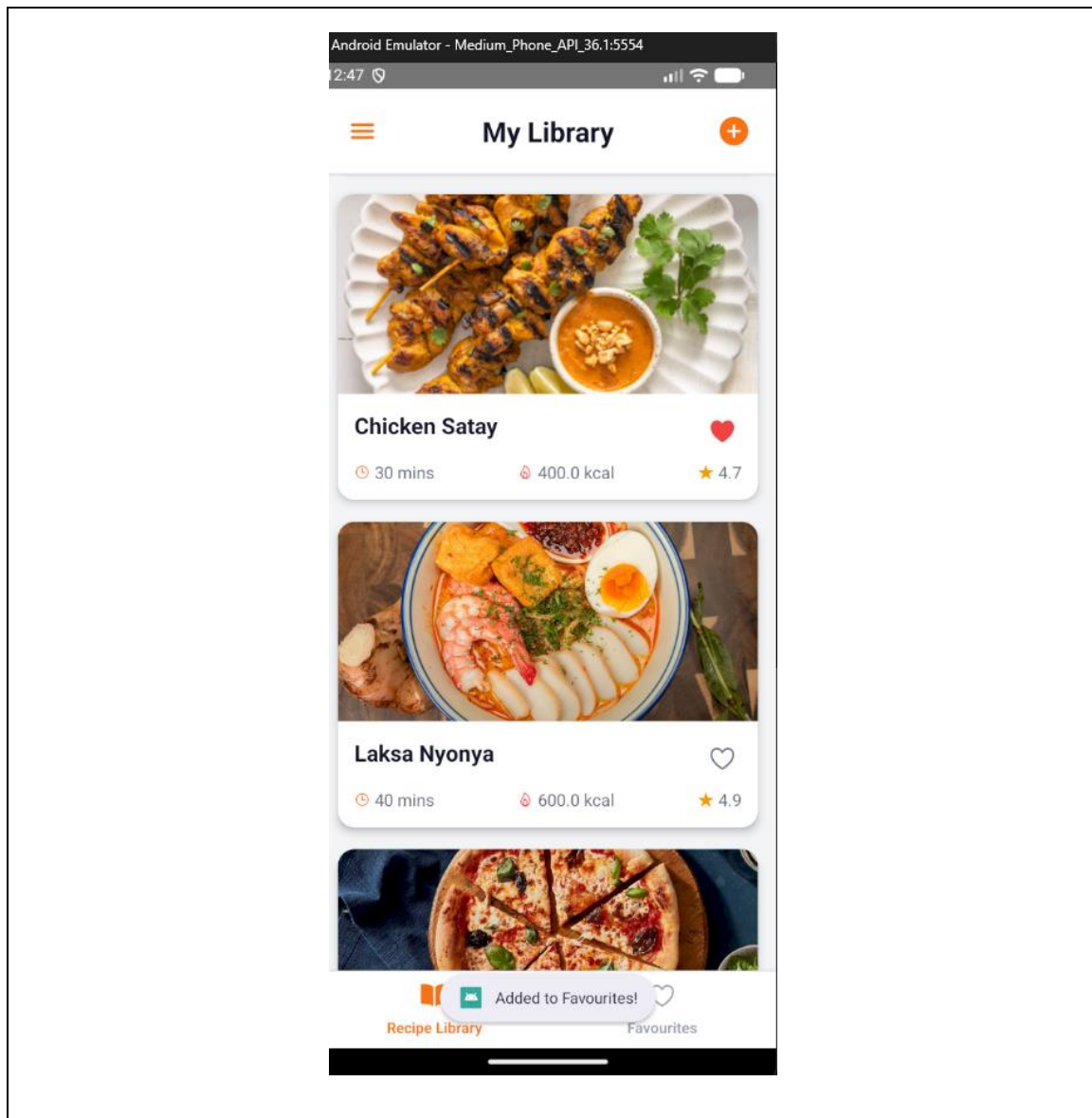
export const _getFavourites = async (): Promise<number[]> => {
  const jsonValue = await AsyncStorage.getItem(FAV_KEY);
  return jsonValue != null ? JSON.parse(jsonValue) : [];
};

export const _toggleFavouriteAsync =
```

```

    async (id: number): Promise<boolean> => {
    let favs = await _getFavourites();
    const isFav = favs.includes(id);
    if (isFav) { favs = favs.filter(f => f !== id); }
    else { favs.push(id); }
    await AsyncStorage.setItem(FAV_KEY, JSON.stringify(favs));
    return !isFav;
  };

```



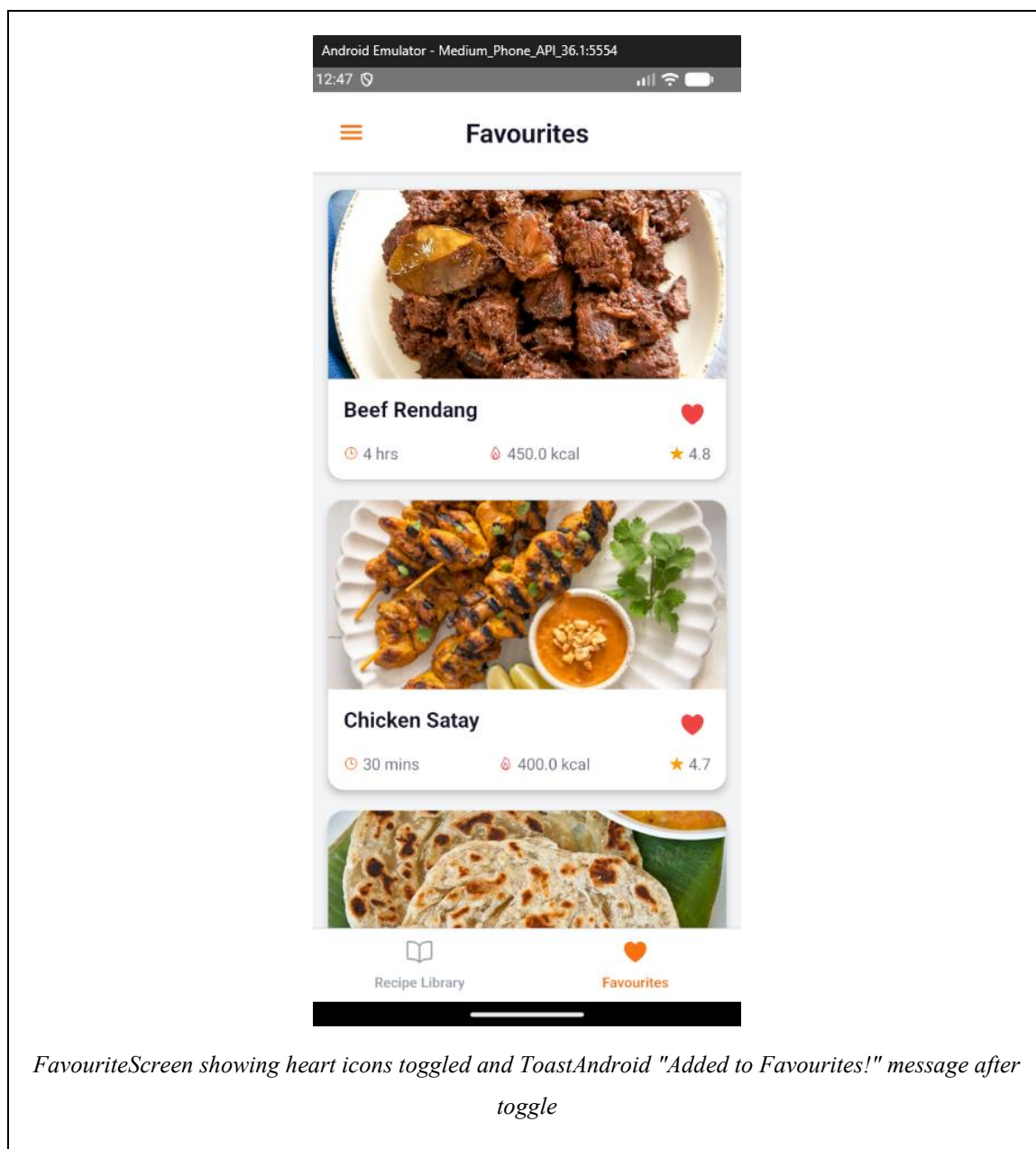


Figure B.2 – AsyncStorage favourite toggle (favourite-service.ts)

File System Backup

The `_exportDataToJson` function queries `SELECT * FROM recipes` from SQLite and reads AsyncStorage to produce a combined JSON payload: `{ recipes: [...], favourites: [...] }`. The `_exportToFileWithName` function calls `RNFS.writeFile` to write this JSON string to `RNFS.DocumentDirectoryPath` with a caller-supplied filename. The `FileBackupScreen` generates the filename as `Recipes_{count}_items_{timestamp}.json`, making it human-readable and uniquely timestamped. The `_importFromFile` function reads the selected file using `RNFS.readFile`, passes its content to `_restoreFromJson`, which first executes `DELETE FROM recipes` to clear all existing records, then iterates the backup recipes array and re-inserts each

record preserving the original id values, and finally calls `AsyncStorage.setItem` to restore the favourites array. The `_listBackupFile` function calls `RNFS.readDir` to list the document directory, filters for '.json' files, and sorts by mtime descending so the newest backup appears first.

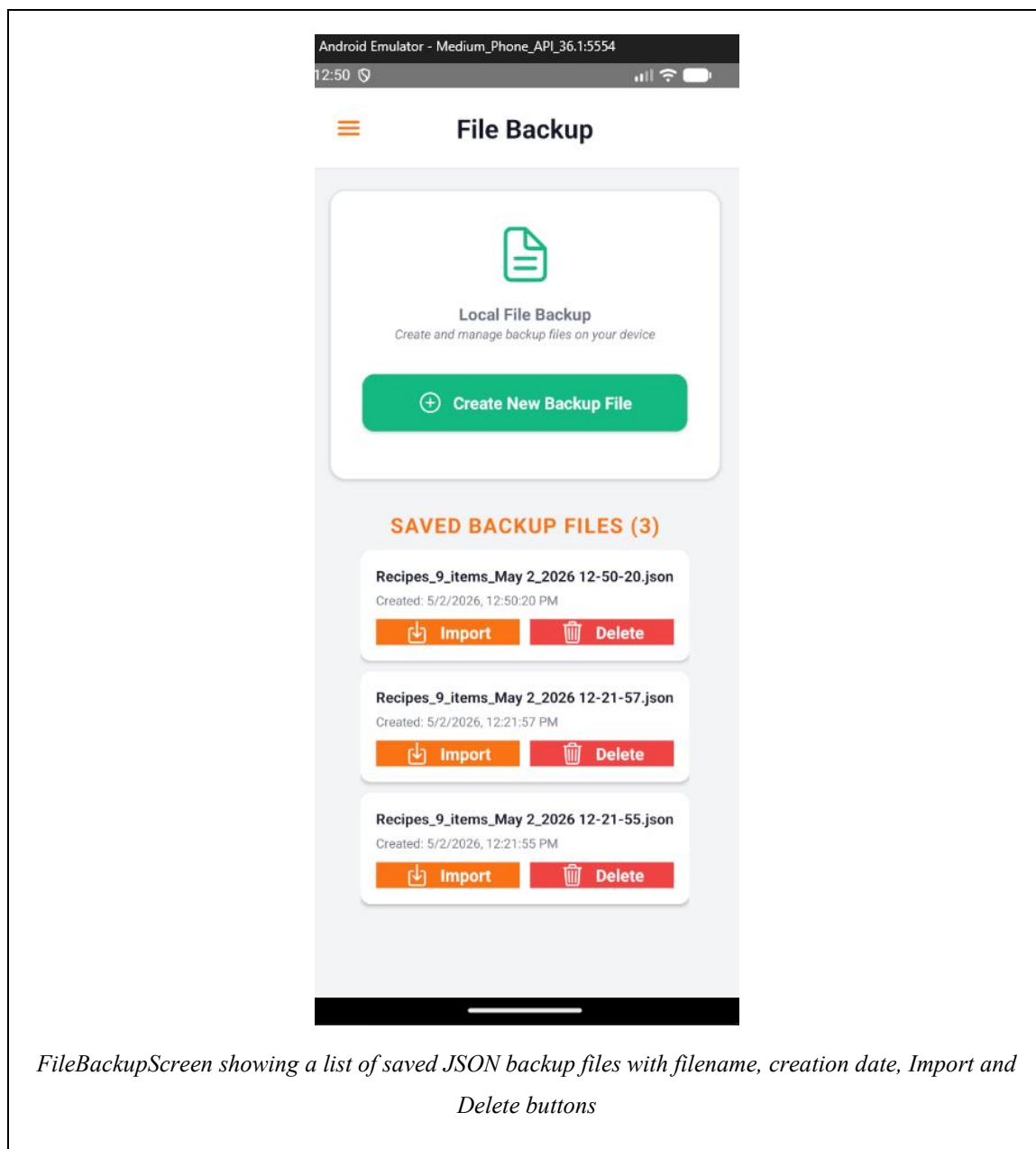
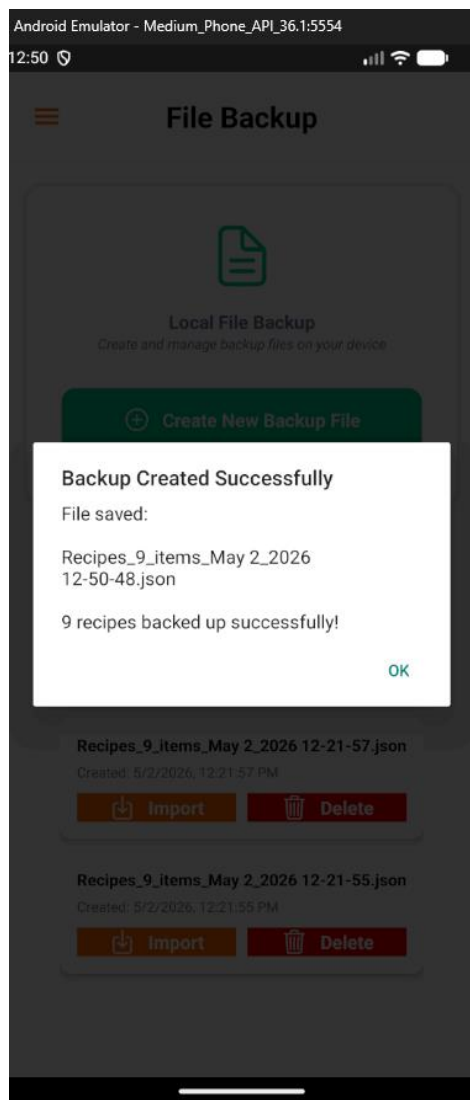
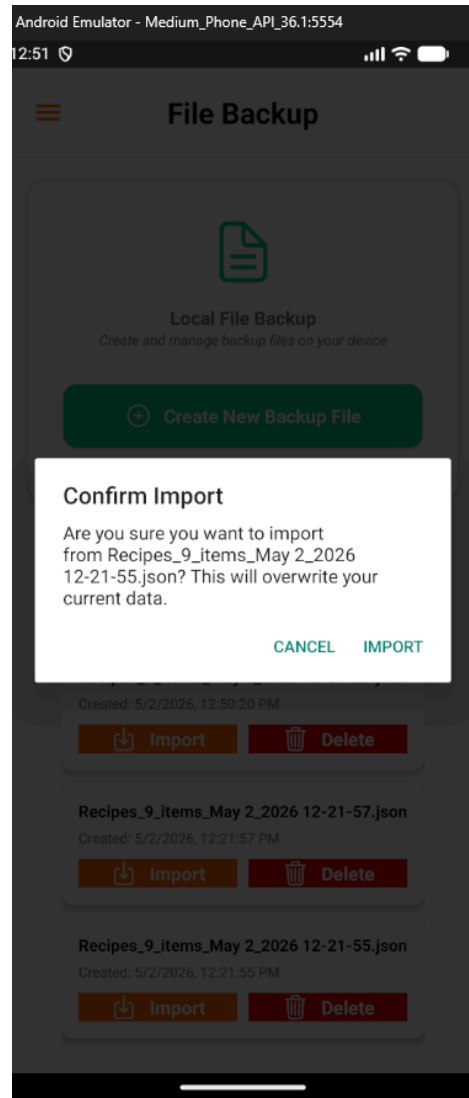


Figure B.3 – File Backup Screen with saved backup files



FileBackupScreen — "Backup Created Successfully" Alert showing generated filename and recipe count after export

Figure B.3b – File backup creation success alert



FileBackupScreen — "Confirm Import" Alert dialog asking user to confirm before overwriting current data

Figure B.3c – File backup import confirmation dialog

B.3 CRUD Operations & Input Validation

Full Create, Read, Update, and Delete (CRUD) operations are implemented for the recipes entity across both the local SQLite database and the cloud server. Input validation is enforced at the UI layer in `AddRecipeScreen` and `EditRecipeScreen` before any database write is attempted. The same seven validation rules are applied identically in both screens using a shared validation block, ensuring consistent data quality across all create and update operations.

Input Validation Rules

The following seven validation checks are performed in sequence when the user submits the form. If any check fails, a `ToastAndroid.SHORT` message is displayed immediately, and the function returns early without calling the database. This immediate, non-blocking feedback mechanism allows users to correct mistakes without losing other entered data.

Field	Rule	Toast Message	Notes
Title	<code>form.title.trim()</code> must not be empty	"Recipe Title is required"	<code>trim()</code> ignores whitespace-only entries
Prep Time	<code>form.prep_time.trim()</code> must not be empty	"Prep Time is required (e.g., 15 mins)"	Format hint included in message
Description	<code>form.description.trim()</code> must not be empty	"Description is required"	
Ingredients	<code>form.ingredients.trim()</code> must not be empty	"Ingredients are required"	
Instructions	<code>form.instructions.trim()</code> must not be empty	"Cooking Instructions are required"	
Calories	<code>parseFloat</code> must be non-NaN and > 0	"Please enter valid Calories (> 0)"	Rejects empty, text, zero, negatives
Rating	<code>parseFloat</code> must be non-NaN and 0 to 5	"Please enter a valid rating (0-5)"	Rejects out-of-range e.g. 5.5, -1

If all seven validations pass, an `Alert.alert` confirmation dialog is shown before the database write occurs. This two-step confirmation, first validating inputs via toast, then requiring explicit user consent via `Alert`, prevents both invalid data entry and accidental submissions.

Creating a Recipe with Invalid Input

When the user taps CREATE without filling in required fields, validation runs immediately and sequentially:

- Title empty → toast "Recipe Title is required" → return (no DB write)
- Prep Time empty → toast "Prep Time is required (e.g., 15 mins)" → return
- Description empty → toast "Description is required" → return
- Ingredients empty → toast "Ingredients are required" → return
- Instructions empty → toast "Cooking Instructions are required" → return
- Calories ≤ 0 or non-numeric → toast "Please enter valid Calories (> 0)" → return
- Rating < 0 or > 5 or non-numeric → toast "Please enter a valid rating (0-5)" → return
- Only if ALL pass → Alert confirmation dialog → saveToDatabase()

The same seven rules apply identically in EditRecipeScreen (handleSave function).

Android Emulator - Medium_Phone_API_36.1:5554

12:52

×

New Recipe

CREATE

RECIPE NAME 0/100

e.g. Nasi Lemak

PREP TIME 0/20

e.g. 30 mins

IMAGE URL 0/500

https://example.com/image.jpg

RATING (0-5)

0.0 to 5.0

DESCRIPTION 0/1000

Fragrant coconut rice served with spicy sambal, anchovies, peanuts, and boiled egg

INGREDIENTS 0/2000

eg. Rice, Coconut Milk, Pandan, Anchovies, Peanuts, Chili, Egg

INSTRUCTIONS 0/5000

eg. 1. Cook rice with coconut milk and pandan.

CALORIES (KCAL)

Recipe Title is required

AddRecipeScreen — ToastAndroid "Recipe Title is required" when Title field is empty and CREATE is tapped

Figure B.4 – Input validation toast: empty Title field

Android Emulator - Medium_Phone_API_36.1:5554

12:55

✕ New Recipe CREATE

Nasi Lemak

PREP TIME 7/20

30 mins

IMAGE URL 0/500

https://example.com/image.jpg

RATING (0-5)

4

DESCRIPTION 23/1000

Rice served with sambal

INGREDIENTS 33/2000

Rice, Coconut Milk, Pandan, Chili

INSTRUCTIONS 41/5000

1. Cook rice with coconut milk and pandan

CALORIES (KCAL)

Enter calories

Please enter valid Calories (> 0)

AddRecipeScreen — ToastAndroid "Please enter valid Calories (> 0)" when Calories field is invalid

Figure B.4b – Input validation toast: invalid Calories

Android Emulator - Medium_Phone_API_36.1:5554
12:58

✕ **New Recipe** CREATE

Nasi Lemak

PREP TIME 7/20
30 mins

IMAGE URL 0/500
https://example.com/image.jpg

RATING (0-5)
7

DESCRIPTION 23/1000
Rice served with sambal

INGREDIENTS 33/2000
Rice, Coconut Milk, Pandan, Chili

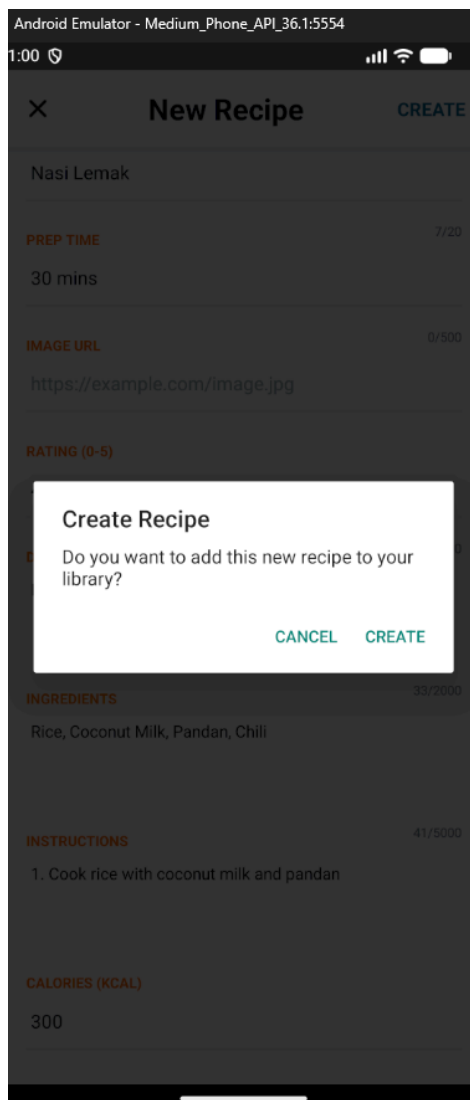
INSTRUCTIONS 41/5000
1. Cook rice with coconut milk and pandan

CALORIES (KCAL)
300

Please enter a valid rating (0-5)

AddRecipeScreen — ToastAndroid "Please enter a valid rating (0-5)" when Rating is out of range

Figure B.4c – Input validation toast: invalid Rating



AddRecipeScreen — "Create Recipe" confirmation Alert shown only after all seven validations pass

Figure B.4d – Create confirmation dialog after validation passes

Create — _insertRecipe

New recipes are created via AddRecipeScreen. The form state is managed by a single useState object with eight string fields initialised as empty strings. After validation and user confirmation, saveToDatabase calls `_insertRecipe` with all fields as parameterised arguments. The `image_url` accepts null if the URL field is left blank (`form.image_url || null`). On success, a "Recipe Added Successfully!" toast is shown, `route.params.refresh()` reloads the RecipeLibraryScreen list, and `navigation.goBack()` dismisses the screen. A cancel handler checks whether the title or description fields are non-empty before navigating away; if they are, a "Discard Recipe?" confirmation is shown to prevent accidental loss of entered data.

Read — `_queryRecipes`, `_queryRecipeById`, `_queryFavouriteRecipes`

Recipes are read from the database in three distinct scenarios. `RecipeLibraryScreen` calls `_queryRecipes` on mount and each time the screen gains focus (`useIsFocused`), returning all records sorted by title with `is_favourite` enrichment. `FavouriteScreen` calls `_queryFavouriteRecipes`, which performs a `WHERE id IN (?)` filtered query using only the IDs stored in `AsyncStorage`. `ViewRecipeScreen` calls `_queryRecipeById` after navigation to retrieve the full record for the requested id. `EditRecipeScreen` also calls `_queryRecipeById` in its mount `useEffect` to pre-populate the form fields.

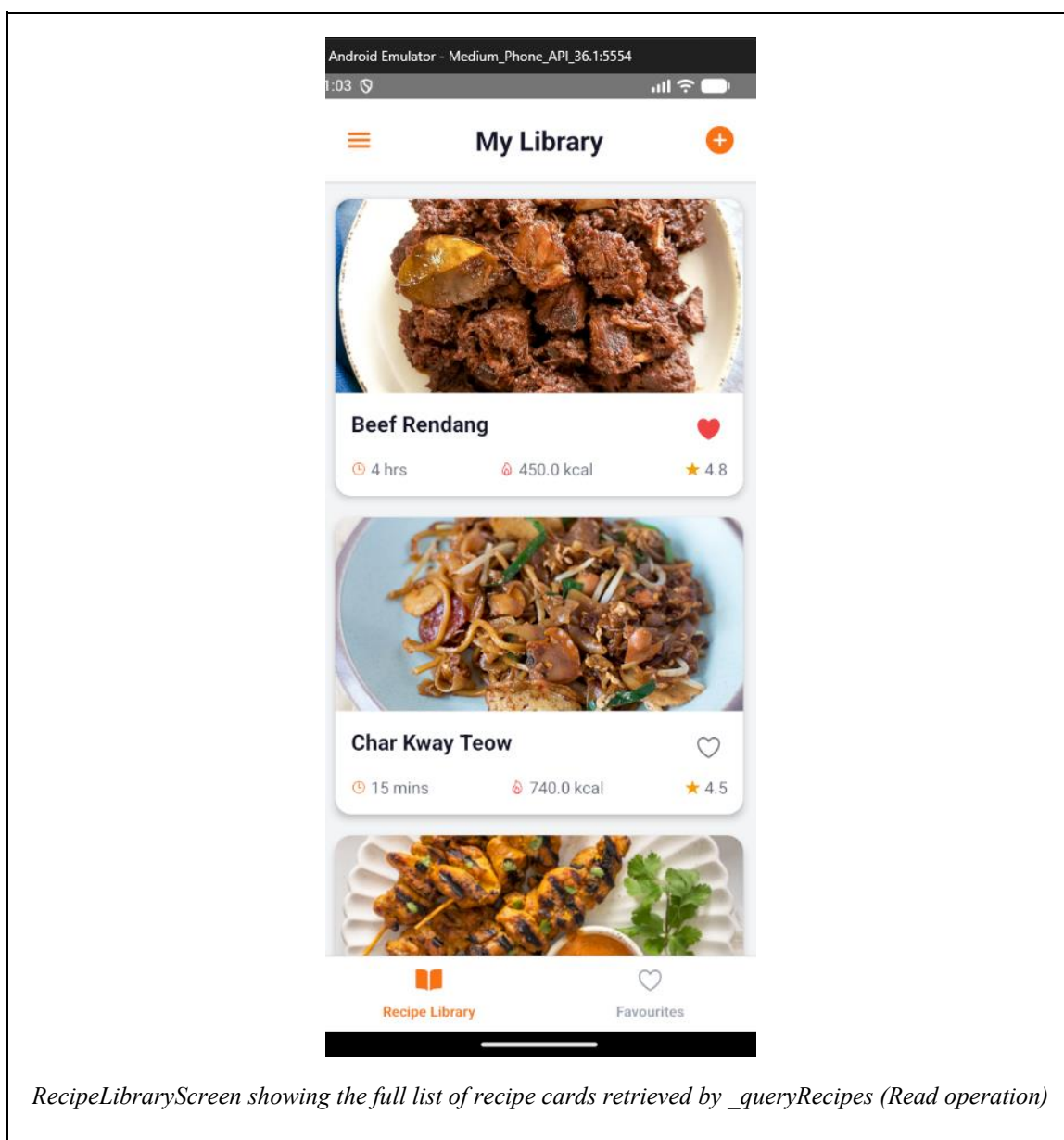


Figure B.5 – Read: `RecipeLibraryScreen` recipe card list

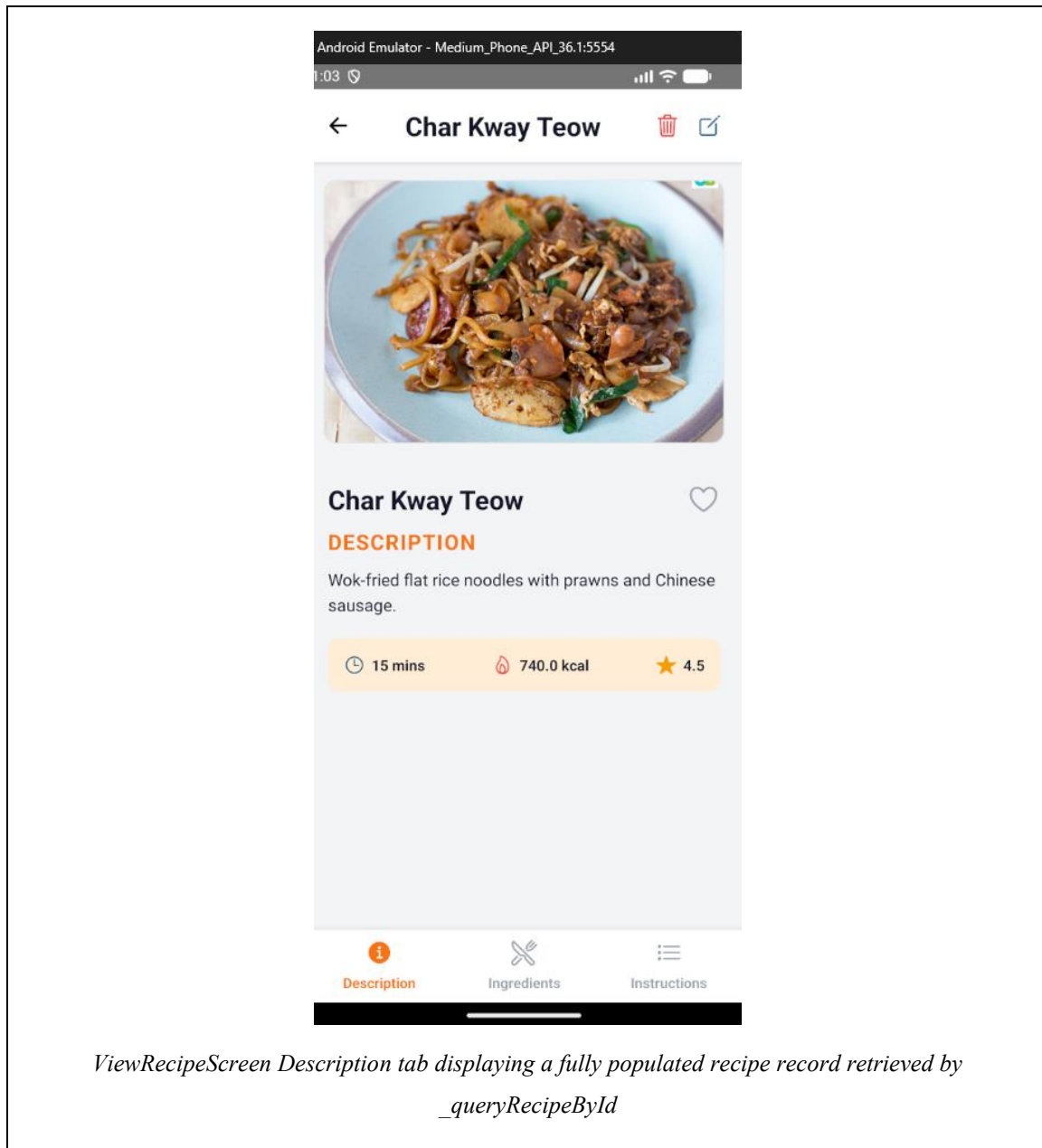
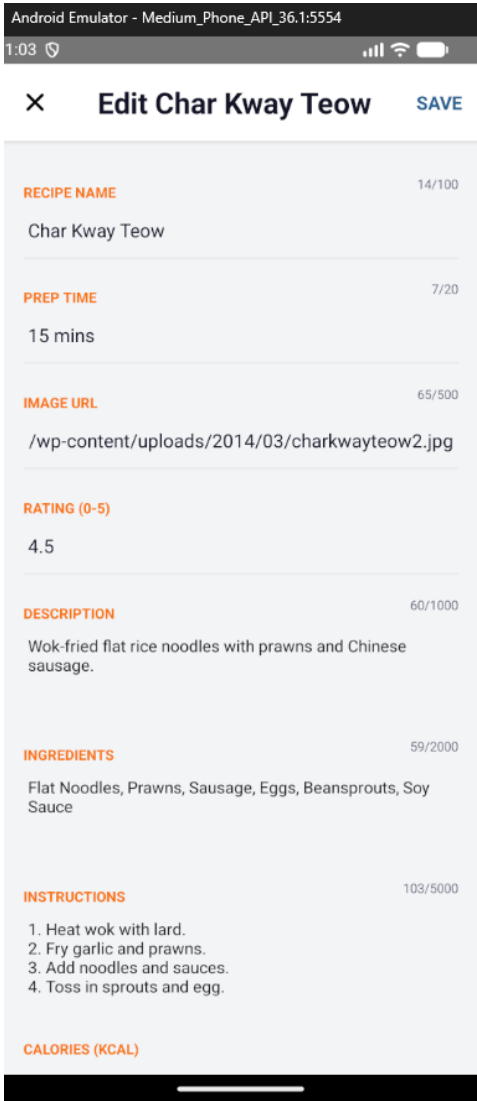


Figure B.5b – Read: single recipe detail view (*ViewRecipeScreen*)

Update — `_updateRecipe`

Existing recipes are updated via `EditRecipeScreen`. The `useEffect` hook on mount calls `_queryRecipeById` and sets the form state with the returned record's field values. The user can modify any field. On tapping "SAVE", `handleSavePress` shows a confirmation `Alert`; on confirming, `handleSave` performs the same seven validation checks as the Create flow. Upon passing, `_updateRecipe` executes `UPDATE recipes SET title=?, description=?, instructions=?, ingredients=?, prep_time=?, calories=?, image_url=?, rating=? WHERE id=?` with nine parameterised arguments. On success, a "Changes Saved!" toast is shown and

`navigation.goBack()` is called. The `refresh` and `homeRefresh` callbacks are both invoked to update the `ViewRecipeScreen` and `RecipeLibraryScreen` data respectively.

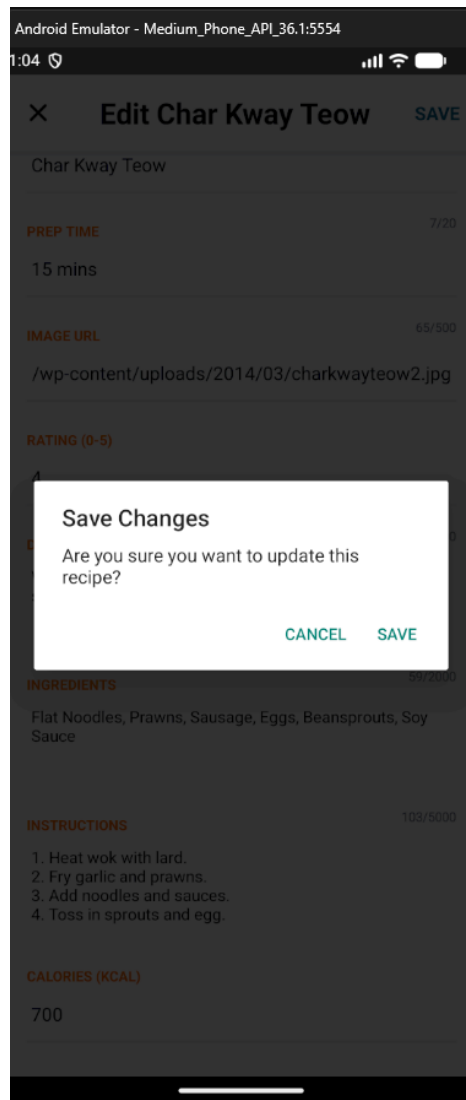


The screenshot displays the 'Edit Char Kway Teow' screen in an Android emulator. The form is pre-filled with the following data:

- RECIPE NAME:** Char Kway Teow (14/100 characters)
- PREP TIME:** 15 mins (7/20 characters)
- IMAGE URL:** /wp-content/uploads/2014/03/charkwayteow2.jpg (65/500 characters)
- RATING (0-5):** 4.5
- DESCRIPTION:** Wok-fried flat rice noodles with prawns and Chinese sausage. (60/1000 characters)
- INGREDIENTS:** Flat Noodles, Prawns, Sausage, Eggs, Beansprouts, Soy Sauce (59/2000 characters)
- INSTRUCTIONS:**
 1. Heat wok with lard.
 2. Fry garlic and prawns.
 3. Add noodles and sauces.
 4. Toss in sprouts and egg.(103/5000 characters)
- CALORIES (KCAL):** (Empty field)

EditRecipeScreen with all form fields pre-populated with the existing recipe data

Figure B.6 – Update: EditRecipeScreen with pre-filled form

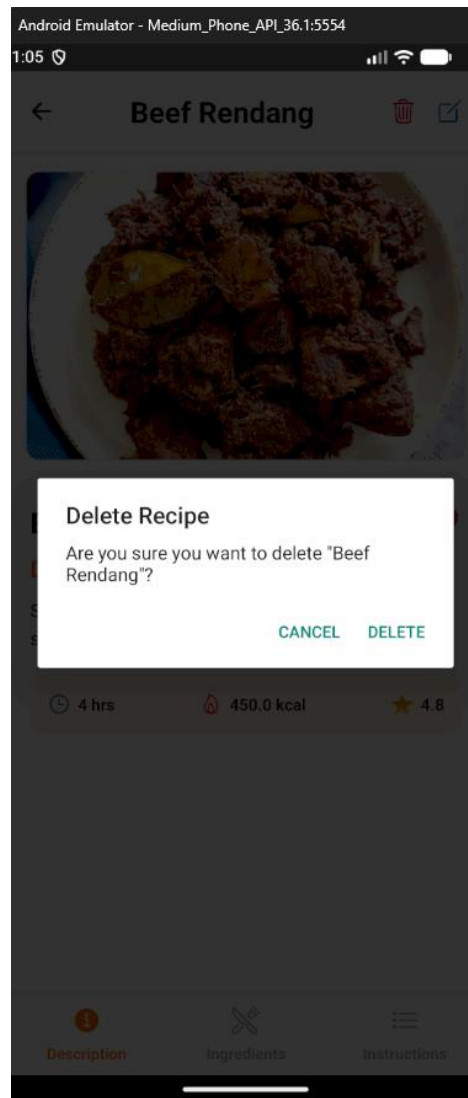


EditRecipeScreen — "Save Changes" confirmation Alert before writing to SQLite

Figure B.6b – Update save confirmation dialog

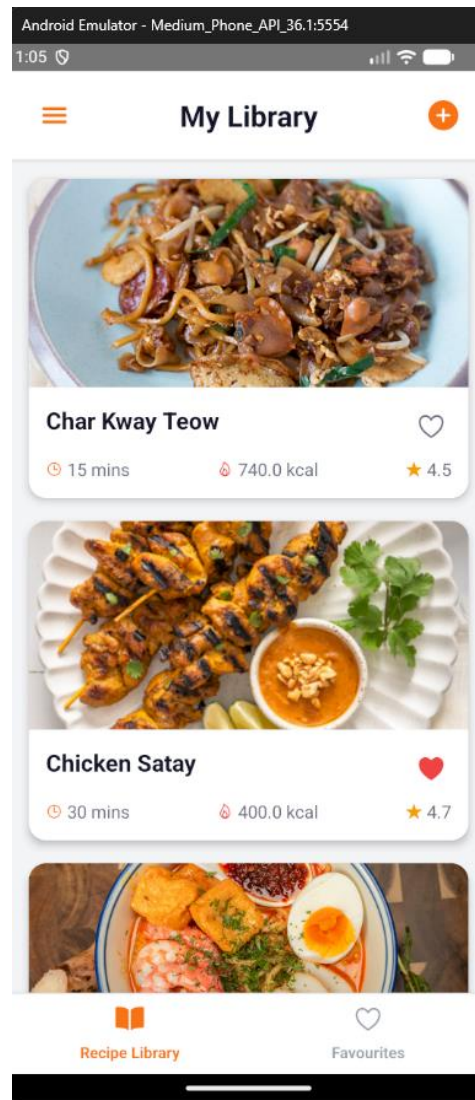
Delete — `_deleteRecipe`

Recipe deletion is initiated from ViewRecipeScreen by tapping the trash icon. The `handleDelete` function shows an `Alert.alert` confirmation dialog displaying the recipe's title: 'Are you sure you want to delete "[title]"?'. On confirming, `_deleteRecipe` executes `DELETE FROM recipes WHERE id = ?` with the parameterised recipe id. On success, a "Recipe Deleted" toast is shown, `route.params.refresh()` updates the calling screen's list, and `navigation.goBack()` returns to the list.



Delete confirmation Alert dialog over ViewRecipeScreen showing the recipe name and Cancel / Delete buttons

Figure B.7 – Delete confirmation dialog



RecipeLibraryScreen after deletion — showing the updated list with the deleted recipe no longer present

Figure B.7b – Recipe Library after successful deletion

C CLOUD CONNECTIVITY

C.1 Appropriate Usage of Cloud Connectivity

Cloud connectivity in the Recipe Manager App is established through two mechanisms: a RESTful Web API for point-in-time data queries and CRUD operations, and a WebSocket connection using the Socket.IO library for the primary real-time backup and restore workflow. Both mechanisms connect to a custom Node.js Express server running on port 5000. On the Android Emulator, the IP address 10.0.2.2 routes requests to the host machine where the server is running, as this is the standard Android Emulator loopback address for the host.

According to UECS3253 Course Notes (2026), REST (Representational State Transfer) is an architectural style proposed by Roy Fielding in his 2000 doctoral dissertation. It uses standard HTTP methods to operate on resources identified by URLs: GET for retrieval, POST for creation, PUT for full replacement, DELETE for removal. REST is stateless, each request contains all information needed to process it, with no session state maintained between requests. The Recipe Manager cloud server exposes the following REST endpoints:

Endpoint	Description
GET /api/recipes	Returns the full array of recipes stored in the cloud backup.
GET /api/recipes/:id	Returns a single recipe by numeric id. Returns HTTP 404 if not found.
POST /api/recipes	Appends a new recipe to the cloud backup with an auto-assigned id. Returns HTTP 201.
PUT /api/recipes/:id	Validates body id matches URL param and replaces the matching recipe. Returns HTTP 200.
DELETE /api/recipes/:id	Validates body id matches URL param and removes the recipe. Returns HTTP 200.
GET /api/backup/status	Returns the timestamp of the last successful cloud backup. Used by CloudSyncScreen at mount.
POST /api/backup	Accepts a full data payload and overwrites cloud_backup.json. Also broadcasts a sync_notification event.
GET /api/restore	Returns the full cloud_backup.json object for polling-based restore.

According to UECS3253 Course Notes (2026), WebSockets are a TCP-based protocol that enables full-duplex persistent communication between a client and a server. Unlike HTTP, which requires the client to initiate every exchange, a WebSocket connection remains open after the initial handshake, , allowing the server to push data to the client at any time. Socket.IO builds on the WebSocket protocol and adds automatic reconnection, namespace support, and event-based messaging (Socket.IO, 2024). UECS3253 Course Notes (2026) lists data synchronisation as a primary WebSocket use case, which is exactly the scenario used in this app.

WebSocket communication via Socket.IO is chosen for the primary backup and restore workflow rather than a simple REST POST/GET pair because it provides real-time, reactive feedback during what can be a significant data transfer. The server can push an acknowledgement with the recorded timestamp immediately after writing the backup file and can push the full backup payload to the client during restore without requiring a polling mechanism or a second HTTP round-trip. The REST API is used for the simpler, stateless operations: querying individual recipe records and fetching the backup status timestamp at app mount time.

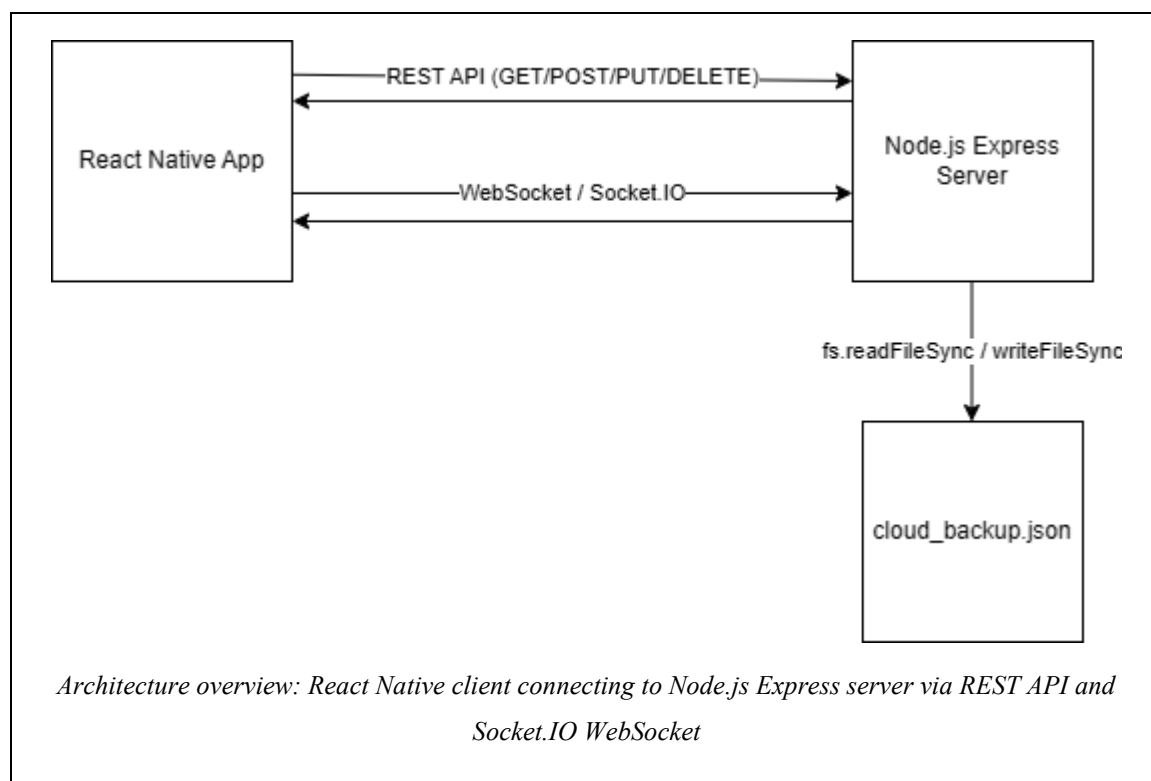


Figure C.0 – Cloud connectivity architecture overview

C.2 Implementation of Cloud Connectivity

The cloud connectivity implementation is split between the server-side `server.js` file and the client-side `CloudSyncScreen.tsx` file. The server uses the Express framework for HTTP routing and the Socket.IO Server library for WebSocket event management. The client uses the `socket.io-client` library for WebSocket connectivity and the JavaScript Fetch API for REST calls.

Server-Side Implementation

The server is initialised by creating an Express app instance, wrapping it with `http.createServer` to produce a raw Node.js HTTP server, and then attaching a Socket.IO Server instance using `new Server(server)`. This architecture allows both Express HTTP routes and Socket.IO WebSocket handlers to share port 5000. The `express.json({ limit: '50mb' })` middleware is applied to handle large JSON payloads from backup operations. The `cloud_backup.json` file in the server directory serves as the server-side data store, containing a timestamp field and a data object with recipes and favourites arrays. All REST and WebSocket operations read from and write to this file using Node.js built-in `fs.readFileSync` and `fs.writeFileSync` calls.

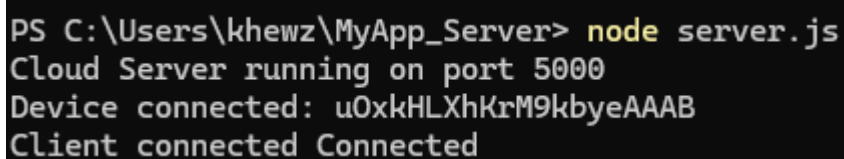
Socket.IO event handlers are registered inside `io.on('connection', (socket) => {...})`. Two custom events are handled per socket: `client_send` with action 'backup' (the server wraps `data.payload` with the provided timestamp, serialises as JSON, and writes it to `cloud_backup.json`, then emits `server_send` with `success: true` back to the same socket), and `client_send` with action 'restore' (the server reads `cloud_backup.json` and emits `server_send` with the full backup data). All REST routes include input validation via the `validateRecipe` helper function before writing data.

```
// server.js - WebSocket backup & restore handlers
io.on('connection', (socket) => {
  socket.on('client_send', (data) => {
    if (data.action === 'backup') {
      const backupData = {
        timestamp: data.timestamp || new Date().toLocaleString(),
        data: data.payload
      };
      fs.writeFileSync(BACKUP_FILE, JSON.stringify(backupData, null, 2));
      socket.emit('server_send', {
        success: true,
        message: 'Backup saved successfully!',
        timestamp: backupData.timestamp
      });
    }
  });
});
```

```

    });
  } else if (data.action === 'restore') {
    if (fs.existsSync(BACKUP_FILE)) {
      const backup = JSON.parse(fs.readFileSync(BACKUP_FILE));
      socket.emit('server_send', { success: true, backup });
    } else {
      socket.emit('server_send', {
        success: false, message: 'No backup found on cloud.'
      });
    }
  }
}
});
});

```



```

PS C:\Users\khewz\MyApp_Server> node server.js
Cloud Server running on port 5000
Device connected: uOxkHLXhKrM9kbyeAAAB
Client connected Connected

```

Terminal output of Node.js server showing socket connection

Figure C.1 – Socket.IO backup and restore handlers (server.js)

Client-Side Implementation

The Socket.IO client is instantiated at module scope in CloudSyncScreen.tsx using `const socket = io('http://10.0.2.2:5000', { transports: ['websocket'] })`. Module-scope instantiation ensures exactly one socket instance exists for the app's lifetime, preventing duplicate connections on component re-renders. The `transports: ['websocket']` option instructs Socket.IO to use the WebSocket transport directly, bypassing the HTTP long-polling fallback that can cause issues in React Native environments (UECS3253 Course Notes, 2026).

The component maintains four state variables: `lastBackupTime` (displayed timestamp), `loadingBackup` and `loadingRestore` (boolean flags replacing buttons with `ActivityIndicator` spinners during operations), and `connectionStatus` (a human-readable string showing the current connection state). In the `useEffect` hook, the component registers four Socket.IO event listeners: `connect` (sets `connectionStatus` to "Connected to Cloud" and calls `fetchBackupStatus`), `disconnect` (sets `connectionStatus` to "Disconnected from Cloud"), `connect_error` (sets `connectionStatus` to "Server Offline"), and `server_send` (updates `lastBackupTime` and

connectionStatus from response data). The useEffect cleanup function calls socket.off() for each registered event to prevent memory leaks.

The handleBackup function guards against offline state by checking socket.connected before proceeding. If connected, it sets loadingBackup: true, calls _exportDataToJson to serialise all local SQLite and AsyncStorage data, generates a timestamp with _generateTimestamp, and emits the client_send backup event. A socket.once('server_send', ...) one-time listener handles the server's acknowledgement. On success, a "Backup Successful" Alert is shown and fetchBackupStatus refreshes the displayed timestamp. The handleRestore function first shows a confirmation Alert, then emits the client_send restore event. If successful, it calls _restoreFromJson(JSON.stringify(response.backup.data)) which wipes the local SQLite recipes table, re-inserts all recipes from the backup, and restores the favourites in AsyncStorage, then calls route.params.refresh() to reload the RecipeLibraryScreen.

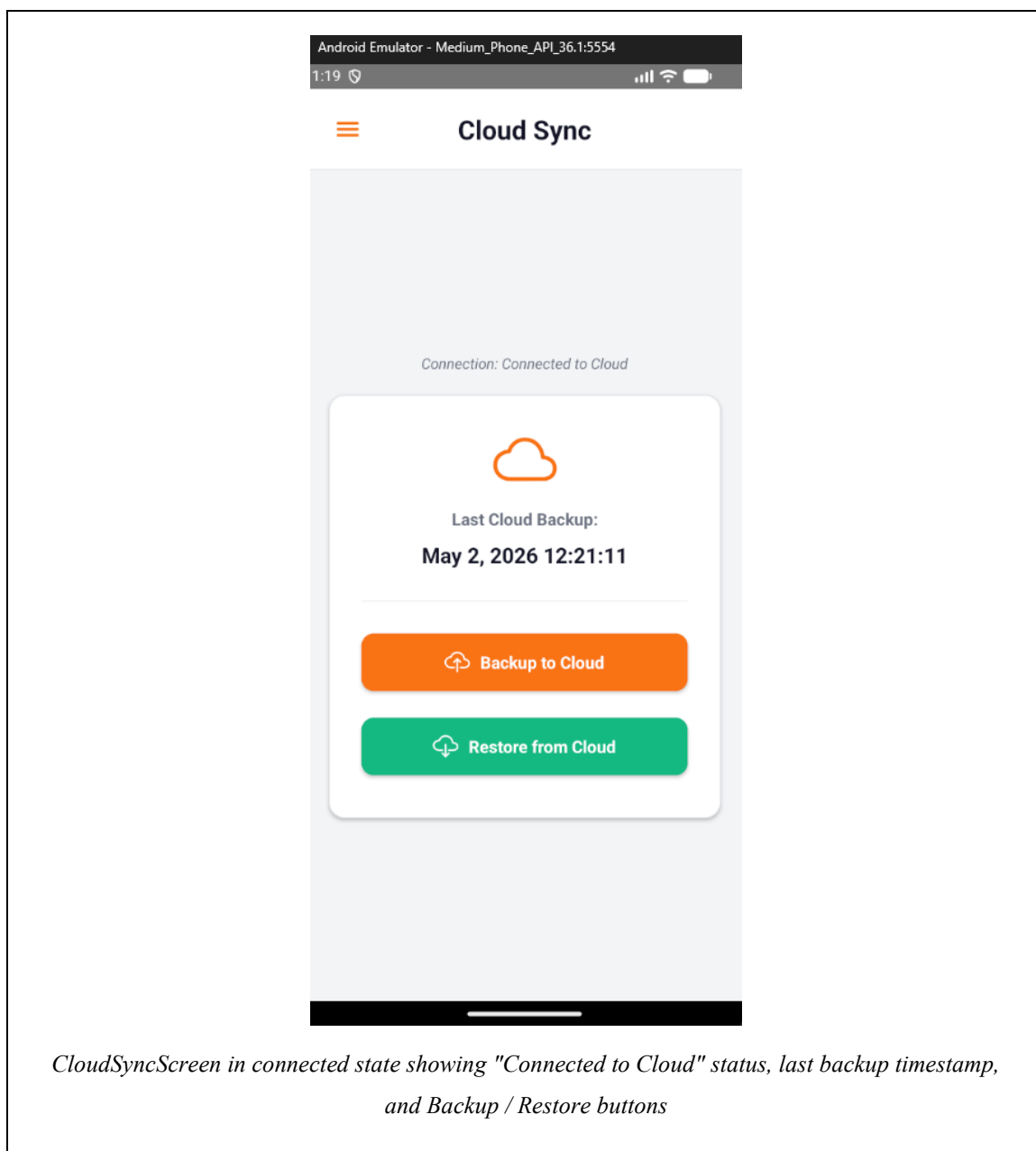


Figure C.2 – CloudSyncScreen connected state

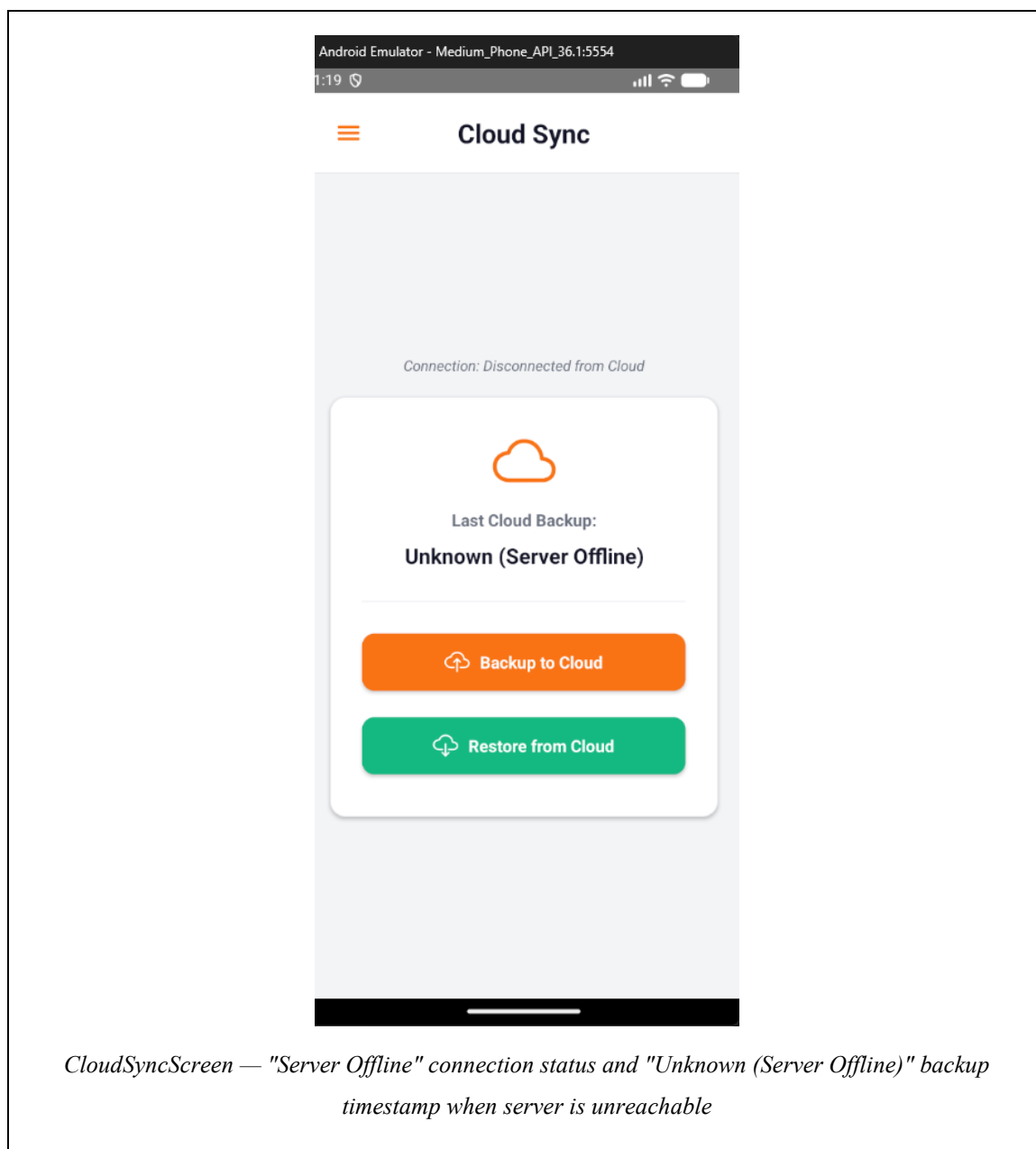
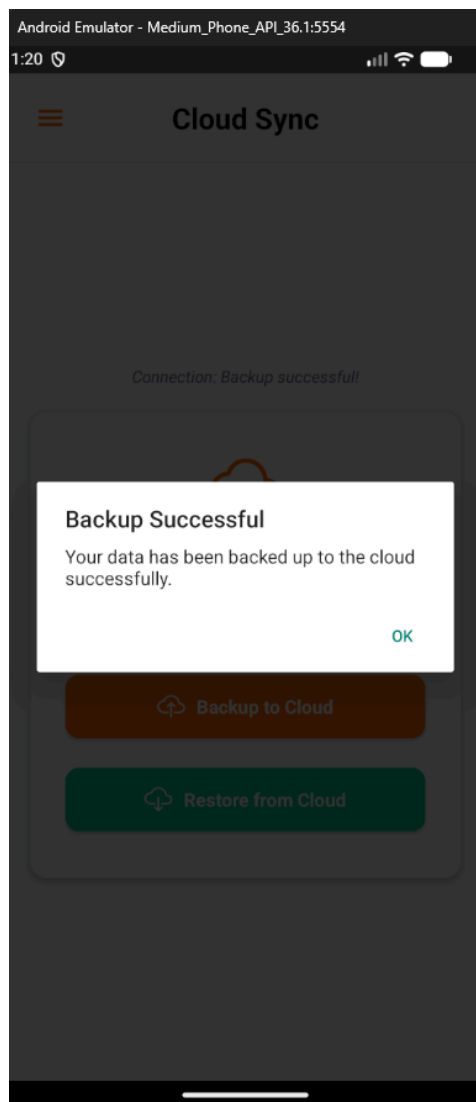


Figure C.2b – CloudSyncScreen server offline state



CloudSyncScreen showing "Backup Successful" Alert popup after a successful WebSocket backup operation

Figure C.3 – Cloud backup success confirmation

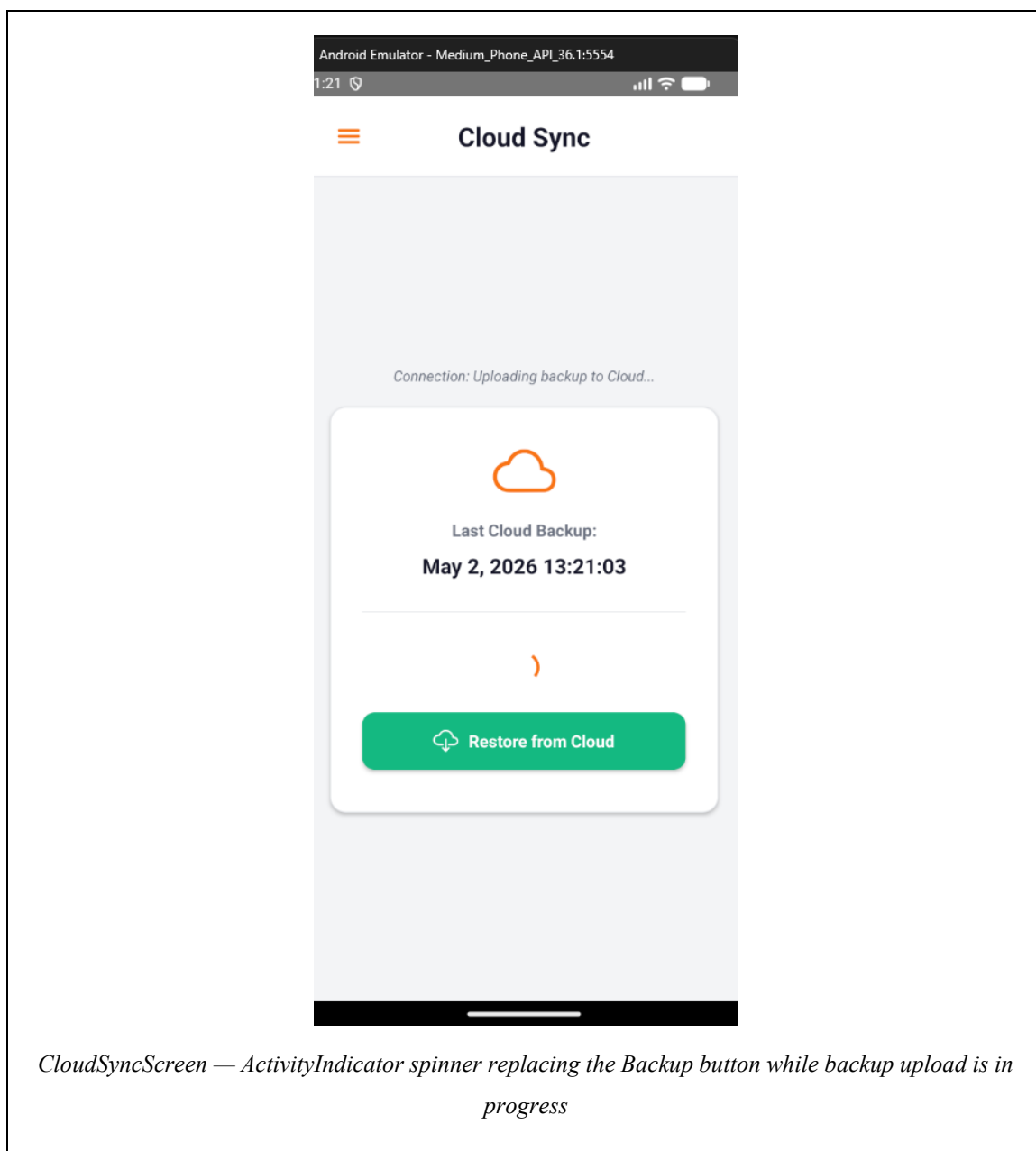
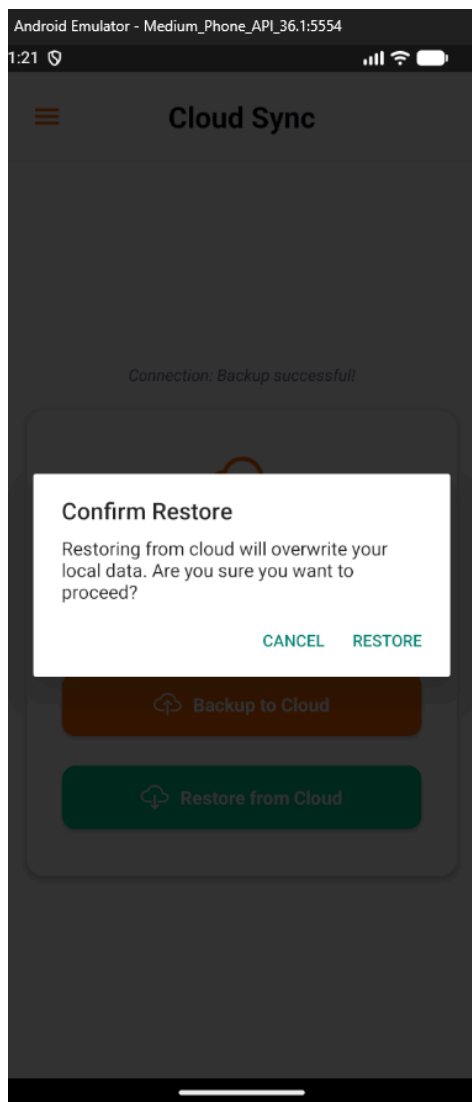
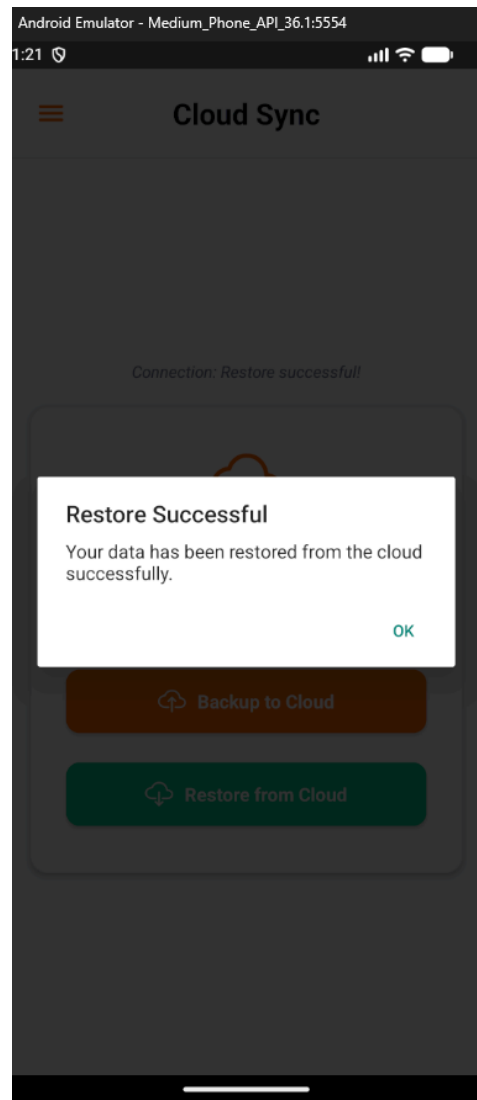


Figure C.3b – CloudSyncScreen backup in-progress indicator



CloudSyncScreen showing the "Confirm Restore" Alert dialog warning that local data will be overwritten

Figure C.4 – Cloud restore confirmation dialog



CloudSyncScreen — "Restore Successful" Alert dialog after cloud data has been fully restored to local SQLite

Figure C.4b – Cloud restore success confirmation

REFERENCES

- Oblador (2026) react-native-vector-icons. Available at: <https://github.com/oblador/react-native-vector-icons> (Accessed: 29 April 2026).
- OpenJS Foundation (2026) Node.js. Available at: <https://nodejs.org/en> (Accessed: 30 April 2026).
- React Native (2026A) ActivityIndicator. Available at: <https://reactnative.dev/docs/activityindicator> (Accessed: 28 April 2026).
- React Native (2026B) Core components and APIs. Available at: <https://reactnative.dev/docs/components-and-apis> (Accessed: 28 April 2026).
- React Native (2026C) FlatList. Available at: <https://reactnative.dev/docs/flatlist> (Accessed: 28 April 2026).
- React Navigation (2026) Getting started. Available at: <https://reactnavigation.org/docs/getting-started> (Accessed: 29 April 2026).
- Socket.IO (2024) Socket.IO documentation. Available at: <https://socket.io/docs/v4/> (Accessed: 29 April 2026).
- SQLite Consortium (2025) About SQLite. Available at: <https://www.sqlite.org/about.html> (Accessed: 29 April 2026).
- UECS3253 Course Notes (2026) 'Wireless Application Development' [Lecture notes].
UECS3253: Wireless Application Development. Universiti Tunku Abdul Rahman, Kampar. Trimester January 2026.
- Wieruch, R. (2023) The minimal React Native navigation. Available at: <https://reactnavigation.org/docs/getting-started> (Accessed: 27 April 2026).

END OF DOCUMENT

PROJECT LINK:

<https://drive.google.com/drive/folders/1Hr8-SP9o8okqiDavSUvTsnmD1zteVLhR?usp=sharing>

Extra installation used but not mentioned in lecture notes:

`npm install react-native-fs`